

Středoškolská odborná činnost

Obor č. 18: Informatika

Interaktivní symbolická regrese využívající AI

Šimon Surýnek
Jihomoravský kraj

Brno 2026

Středoškolská odborná činnost

Obor č. 18: Informatika

Interaktivní symbolická regrese využívající AI

Interactive symbolic regression using AI

Autor: Šimon Surýnek

Škola: Gymnázium Brno-Řečkovice, příspěvková organizace, Terezy Novákové 2, Brno 621 00

Kraj: Jihomoravský kraj

Konzultant Ing. Vojtěch Mrázek, Ph.D.

Brno 2026

Prohlášení

Prohlašuji, že jsem svou práci SOČ vypracoval samostatně a použil jsem pouze prameny a literaturu uvedené v seznamu bibliografických záznamů.

Beru na vědomí, že nejpozději odevzdáním slovesné vědecké práce do veřejné soutěže Středoškolská odborná činnost, stejně jako odevzdáním jejích příloh a dalších připojených děl, např. audiovizuálních, fotografických, výtvarných, architektonických apod. (dále jen „soutěžní dílo“), dochází ke zveřejnění díla podle § 4 odst. 1 zákona č. 121/2000 Sb., autorského zákona, ve znění pozdějších předpisů (dále jen „autorský zákon“). Totéž platí pro pozdější odevzdání doplněného, změněného, upraveného nebo opraveného díla.

Beru na vědomí, že zveřejněním díla, jehož součástí je vynález, se tento vynález stává součástí stavu techniky podle § 5 odst. 1, 2 zákona č. 527/1990 Sb., o vynálezech, průmyslových vzorech a zlepšovacích návrzích, ve znění pozdějších předpisů (dále jen „patentový zákon“), což zakládá překážku pro udělení patentu podle § 3 odst. 1 patentového zákona.

Beru na vědomí, že vyhlašovatel soutěže je podle § 61 odst. 1 autorského zákona per analogiam oprávněn užít soutěžní dílo pro účely zajištění průběhu soutěže, zejména k zajištění transparentnosti soutěže a veřejnosti obhajob soutěžních prací. V odůvodněném rozsahu je tedy vyhlašovatel po dobu účasti autora v soutěži oprávněn zejména:

- zhotovovat rozmnoženiny díla, je-li to nezbytné k seznámení účastníků soutěže, porotců nebo veřejnosti se soutěžní prací;
- zapůjčit originál nebo rozmnoženinu díla účastníkům soutěže, porotcům nebo veřejnosti. Přitom dbá na bezpečné nakládání s dílem;
- vystavovat originál nebo rozmnoženinu díla v průběhu soutěžních přehlídek a doprovodných akcí;
- sdělovat dílo veřejnosti v nehmotné podobě, a to především počítačovou nebo obdobnou sítí.

Dále prohlašuji, že při tvorbě této práce jsem použil nástroj generativního modelu AI [Claude; claude.ai] za účelem [kontroly gramatiky, stylistické úpravy a reformulace textu, debugging kódu]

V Brně dne 30. 1. 2026

Šimon Surýnek

Poděkování

Rád bych poděkoval mému externímu konzultantovi Ing. Vojtěchu Mrázkovi, Ph.D. z Fakulty informačních technologií VUT v Brně za odborné konzultace, cenné připomínky k implementaci algoritmu a trpělivé zodpovídání mých dotazů týkajících se kartézského genetického programování.

Děkuji také prof. Ing. Lukáši Sekaninovi, Ph.D. za inspirativní diskuze o evolučních algoritmech, které mě motivovaly k zpracování tohoto tématu.

Za korekturu práce a kontrolu formální stránky bych chtěl poděkovat mé interní konzultantce Mgr. Martině Blahové.

Tato práce byla vypracována za finanční podpory JMK.



Anotace

Tato práce se zabývá implementací interaktivní webové aplikace pro symbolickou regresi využívající kartézské genetické programování (CGP). Aplikace umožňuje uživatelům experimentovat s parametry evolučního algoritmu a sledovat průběh hledání matematických vztahů v reálném čase. Teoretická část práce popisuje principy evolučních algoritmů, genetického programování a symbolické regrese. Praktická část detailně dokumentuje implementaci evolučního algoritmu typu $(1+\lambda)$, fitness funkce s penalizací složitosti a tvorbu intuitivního grafického rozhraní. Experimentální vyhodnocení analyzuje vliv klíčových parametrů (λ , počet uzlů a pravděpodobnost mutace) na konvergenci algoritmu, porovnává různé strategie inicializace a testuje práci s fixními i evolujícími se konstantami. Výsledky ukazují, že správné nastavení parametrů výrazně ovlivňuje kvalitu nalezených řešení a schopnost algoritmu předcházet problémům jako je bloat a overfitting. Funkčnost algoritmu byla ověřena na reálném fyzikálním problému šikmého vrhu, kde dosáhl rekonstrukce trajektorie s přesností nad 98 %.

Klíčová slova

kartézské genetické programování, evoluční algoritmy, fitness funkce, optimalizační problém, symbolická regrese, webová aplikace

Annotation

This thesis deals with the implementation of an interactive web application for symbolic regression using Cartesian Genetic Programming (CGP). The application allows users to experiment with evolutionary algorithm parameters and observe the process of discovering mathematical relationships in real time. The theoretical part describes the principles of evolutionary algorithms, genetic programming, and symbolic regression. The practical part provides detailed documentation of the implementation of a $(1+\lambda)$ evolutionary algorithm, a fitness function with complexity penalization, and the development of an intuitive graphical user interface. The experimental evaluation analyzes the influence of key parameters (λ , number of nodes, and mutation probability) on algorithm convergence, compares different initialization strategies, and tests both fixed and evolving constants. The results show that proper parameter configuration significantly affects the quality of discovered solutions and the algorithm's ability to prevent problems such as bloat and overfitting. The algorithm's functionality was validated on a real physical problem of projectile motion, achieving trajectory reconstruction with accuracy exceeding 98 %.

Keywords

cartesian genetic programming, evolutionary algorithms, fitness function, optimization problem, symbolic regression, web application

Obsah

Úvod	10
1 Teoretický základ	12
1.1 Optimalizační problém	12
1.1.1 Symbolická regrese jako optimalizační problém	12
1.2 Základní algoritmy prohledávání	14
1.2.1 Náhodné prohledávání	14
1.2.2 Horolezecký algoritmus	14
1.2.3 Evoluční algoritmy	15
1.3 Evoluční algoritmy	15
1.3.1 Historie a vznik	15
1.3.2 Základní principy	16
1.3.3 Pojmy evolučních algoritmů	18
1.3.4 Genetické programování	20
1.4 Symbolická regrese	22
1.4.1 Metriky vyhodnocování v SR	23
1.4.2 Problémy a výzvy SR	24
1.4.3 Aplikace SR	25
2 Cíle práce a návrh	26
2.1 Cíle práce	26
2.2 Návrh a volba technologií	27
2.2.1 Návrh evolučního algoritmu	27
2.2.2 Návrh fitness funkce	28
2.2.3 Návrh grafického rozhraní	28
2.2.4 Volba technologií	29
3 Implementace webové aplikace	30
3.1 Evoluční algoritmus	31
3.1.1 Hlavní evoluční smyčka	31
3.1.2 Reprezentace chromozomu	32
3.1.3 Inicializace chromozomu	32

3.1.4	Fitness funkce	33
3.1.5	Evaluace chromozomu	34
3.1.6	Mutace	35
3.1.7	Práce s konstantami	36
3.1.8	Generování funkčního předpisu	36
3.2	Grafické rozhraní	37
3.2.1	Struktura rozhraní	37
3.2.2	Vykreslování grafu	38
3.2.3	Načítání dat a reaktivita	38
4	Vyhodnocení výsledků	40
4.1	Vliv parametrů evoluce na konvergenci	40
4.1.1	Parametr lambda	40
4.1.2	Parametr pravděpodobnost mutace(P_m)	41
4.1.3	Parametr COLS (maximální počet uzlů)	42
4.1.4	Porovnání vlivu nastavení konstant	43
4.1.5	Porovnání strategií inicializace	43
4.1.6	Rekonstrukce testovacích datasetů	44
4.2	Reálná aplikace – šikmý vrh	44
5	Závěr a diskuze	46
5.1	Shrnutí dosažených cílů	46
5.2	Zhodnocení výsledků	46
5.3	Omezení práce	47
5.4	Možnosti budoucího rozšíření	47
A	Struktura souboru	
B	Kompletní výsledky	
C	Rekonstrukce testovacích datasetů	

Seznam obrázků

1.1	Problém lokálního a globálního maxima při prohledávání stavového prostoru. Algoritmus může uvíznout v lokálním optimu. Zdroj: [1]	15
1.2	Diagram znázorňující chod evolučního algoritmu (inspirováno [2]) . . .	15
1.3	Schéma ilustrující kompromis mezi explorací a exploatací. Zdroj: [3] . .	17
1.4	Schéma ilustrující způsoby křížení. Zdroj: [4]	19
1.5	Schéma ilustrující stromovou strukturu. Zdroj: [5]	20
1.6	Schéma reprezentace CGP: mřížka uzlů s genotypem (posloupnost celých čísel) a fenotypem (aktivní uzly tvořící výsledný program) Zdroj: [6] . .	22
2.1	Wireframe grafického rozhraní aplikace	28
3.1	Aplikace před spuštěním evoluce.	37
3.2	Aplikace po dokončení evoluce.	37
4.1	Grafy konvergence fitness v závislosti na počtu evaluací pro parametr λ	40
4.2	Boxploty finálních hodnot fitness v závislosti na pravděpodobnosti mutace	41
4.3	Boxploty finální fitness v závislosti na parametru COLS	42
4.4	Grafy konvergence fitness demonstrující vliv nastavení konstant	43
4.5	Grafy konvergence fitness demonstrující vliv typu inicializace	44
4.6	Screenshot aplikace při zrekonstruování šikmého vrhu	45

Úvod

Při zkoumání přírodních jevů, analýze technických systémů nebo studiu ekonomických procesů se vědci a inženýři pravidelně setkávají s problémem, kdy mají k dispozici naměřená data, ale neznají přesný matematický vztah mezi sledovanými veličinami. Například fyzik měří dráhu projektilu v závislosti na čase, biolog sleduje rychlost růstu rostliny v závislosti na teplotě, nebo ekonom analyzuje vztah mezi investicemi a ziskem. Ve všech těchto případech je žádoucí najít matematickou funkci, která data co nejlépe popisuje a umožňuje tak pochopit podstatu jevu a předpovídat jeho budoucí chování.

Klasické regresní metody, jako je lineární nebo polynomická regrese, vyžadují, aby uživatel předem určil typ hledané funkce – například, že vztah bude lineární ($f(x) = ax + b$) nebo kvadratický ($f(x) = ax^2 + bx + c$). Tento přístup však selhává v situacích, kdy skutečný vztah má složitější charakter zahrnující například kombinaci polynomů, goniometrických nebo exponenciálních funkcí. Symbolická regrese tento problém řeší tím, že automaticky objevuje jak strukturu matematického výrazu, tak jeho parametry přímo z dat bez nutnosti předem specifikovat typ funkce.

V současnosti existuje několik nástrojů pro symbolickou regresi, například komerční software *Eureqa* (nyní součást *DataRobot*) nebo open-source knihovny jako *gplearn* a *PySR*. Tyto nástroje však mají určitá omezení. Komerční řešení jsou často drahá a fungují jako černé skříňky, kde uživatel nevidí do průběhu evoluce a nemá kontrolu nad procesem hledání. Open-source knihovny zase vyžadují programátorské znalosti (typicky Python), složité nastavení parametrů a poskytují výsledek až po dokončení výpočtu, bez možnosti sledovat průběžný vývoj nebo experimentovat s parametry v reálném čase.

Tato práce představuje interaktivní webovou aplikaci pro symbolickou regresi s využitím kartézského genetického programování – variantu evolučních algoritmů inspirovaných biologickou evolucí. Hlavním přínosem aplikace je její interaktivita a přístupnost. Aplikace běží přímo ve webovém prohlížeči bez nutnosti instalace, nabízí intuitivní grafické rozhraní pro nastavení parametrů evoluce a umožňuje sledovat průběh hledání matematického vztahu v reálném čase prostřednictvím grafu a logů. Uživatel tak může experimentovat s různými nastaveními (počet potomků, pravděpodobnost mutace, typ konstant, ...) a okamžitě vidět jejich vliv na kvalitu a rychlost konvergence

algoritmu. Aplikace byla úspěšně otestována na pěti různých typech závislostí a na reálném fyzikálním problému šikmého vrhu, kde algoritmus dokázal s vysokou přesností zrekonstruovat původní funkci.

Aplikace má díky vizualizaci evolučního procesu nejen praktický, ale i vzdělávací potenciál, neboť usnadňuje pochopení principů kartézského genetického programování a jeho využití v oblasti strojového učení. Práce je strukturována do pěti kapitol: po teoretickém úvodu a definici cílů následuje popis implementace algoritmu i grafického rozhraní. Klíčovou částí je čtvrtá kapitola, která přináší systematickou analýzu vlivu parametrů na výkon algoritmu a demonstruje jeho schopnost řešit reálný fyzikální problém šikmého vrhu. Závěr práce shrnuje dosažené výsledky a navrhuje možnosti dalšího rozšíření systému.

Zdrojový kód aplikace je veřejně dostupný na platformě GitHub: <https://github.com/SimonSurynek/soc-regrese>.

Aplikace je nasazena online prostřednictvím GitHub Pages a dostupná pod odkazem: (<https://simonsurynek.github.io/soc-regrese/>), kde si ji může kdokoliv vyzkoušet bez nutnosti instalace.

1 Teoretický základ

1.1 Optimalizační problém

Optimalizační úlohy vycházejí z problémů reálného světa, kde je potřeba maximalizovat nebo minimalizovat určité parametry – například maximalizovat zisk nebo minimalizovat náklady. V praxi nacházejí tyto úlohy využití například při hledání nejkratší cesty mezi městy, optimálního nastavení motoru nebo při hledání matematické funkce, která nejlépe popisuje naměřená data.

Optimalizační problém je matematická úloha, ve které hledáme extrém (minimum nebo maximum) předem dané funkce, označované jako cílová (účelová) funkce f_0 . Tato funkce je závislá na jedné nebo více proměnných, které mohou být spojité nebo diskrétní. Cílem je nalézt takové hodnoty proměnných

$$\mathbf{x} = (x_1, x_2, \dots, x_n), \quad \mathbf{x} \in \mathbb{R}^n,$$

kde n označuje počet vstupních proměnných popisujících řešený problém, které vedou k nejlepšímu (optimálnímu) výsledku v rámci zadaných omezení [7].

Rovnici obecné optimalizační úlohy, kde se snažíme minimalizovat cílovou funkci, lze zapsat následovně [8]:

$$\min_{x \in \mathbb{R}^n} f_0(x)$$

Analogicky můžeme zapsat rovnici s cílem maximalizace:

$$\max_{x \in \mathbb{R}^n} f_0(x)$$

Kde:

- $x \in \mathbb{R}^n$ je přípustný vektor rozhodovacích proměnných (*feasible point*), tj. vektor, který splňuje všechna omezení úlohy,
- $f_0(x)$ je cílová (účelová) funkce, kterou chceme minimalizovat nebo maximalizovat

1.1.1 Symbolická regrese jako optimalizační problém

Symbolická regrese (SR) je specifický typ optimalizačního problému, ve kterém hledáme matematickou funkci $f(x)$, která co nejlépe popisuje zadanou množinu datových bodů,

$x[i], y[i], i = 0, 1, \dots, n$. Na rozdíl od klasických regresních metod, kde musíme předem zvolit typ matematického modelu, symbolická regrese automaticky objevuje jak formu, tak parametry výrazu [9].

Cílem je minimalizovat měřítko chyby mezi pozorovanými hodnotami $y[i]$ a hodnotami predikovanými funkcí $\hat{y}[i]$. Nejčastěji se používá střední kvadratická chyba (Mean Squared Error, MSE), podrobněji popsána v kapitole 1.4.1.

Symbolická regrese má oproti klasickým metodám jednu zásadní výhodu – není omezena pevně danou strukturou funkce. Zatímco u lineární regrese ($f(\mathbf{x}) = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$) nebo polynomicke regrese ($f(x) = w_0 + w_1x + w_2x^2 + \dots + w_kx^k$) musíme předem určit, jaký typ vztahu budeme hledat, symbolická regrese sama nachází vhodnou matematickou formu. Tato flexibilita je klíčová zejména v oblastech, kde přesné vztahy mezi měřenými veličinami neznáme. Praktické využití symbolické regrese zahrnuje například automatické objevování fyzikálních zákonů z experimentálních dat, které ověřuje schopnost algoritmu rekonstruovat známé vztahy z dat zatížených šumem, nebo modelování komplexních astrofyzikálních procesů. Astrofyzikální úlohy jsou pro tento účel vhodné, protože mnoho základních vztahů je popsáno analytickými rovnicemi, které lze ze simulovaných nebo pozorovaných dat znovu objevit [10].

Zásadním problémem symbolické regrese je její kombinatorická složitost. Prostor všech možných matematických výrazů je prakticky nekonečný – funkce může být libovolnou kombinací aritmetických operací ($+, -, \times, \div$), matematických funkcí (například $\sin, \cos, \exp, \log$), proměnných a konstant. Složitost tohoto prostoru roste exponenciálně s délkou symbolického výrazu, což činí symbolickou regresi NP-těžkým problémem [11]. Označení NP-těžký znamená, že problém je přinejmenším tak obtížný jako nejtěžší problémy ve třídě NP. Pro NP-těžké problémy není znám polynomiální algoritmus, který by zaručeně našel optimální řešení v rozumném čase [11].

Z tohoto důvodu nelze použít klasické optimalizační metody jako gradientní sestup (iterativní metoda, která vypočítá chybu modelu na trénovacích datech a postupně upravuje parametry ve směru jejího snížení [12]), které jsou účinné pouze při hledání parametrů v předem dané struktuře. Místo toho je nutné prohledávat prostor možných struktur funkcí, což vyžaduje pokročilé heuristické přístupy.

Pro řešení symbolické regrese byly vyvinuty různé přístupy. Mezi nejpoužívanější patří genetické programování (Genetic Programming, GP), které iterativně vyvíjí populaci matematických výrazů pomocí evolučních operátorů jako selekce, křížení a mutace [10]. Další moderní přístupy využívají metody strojového učení, například posilované učení (Reinforcement Learning) nebo transformer-based modely natrénované na rozsáhlých datasetech [9].

1.2 Základní algoritmy prohledávání

Jak bylo uvedeno, symbolická regrese je NP-těžký optimalizační problém s obrovskou množinou možných řešení. Pro efektivní nalezení vhodné funkce byly vyvinuty algoritmy prohledávání, které systematicky procházejí prostor možných řešení a hledají optimum. V následujících podkapitolách budou představeny tři základní přístupy (náhodné prohledávání, horolezecký algoritmus a evoluční algoritmy), které se liší v efektivitě, složitosti a schopnosti najít globální optimum.

1.2.1 Náhodné prohledávání

Algoritmus náhodného prohledávání funguje na principu náhody při hledání optimálního řešení [13]. Používá se především při práci se složitými nekonvexními daty. Je oblíbený pro svoji jednoduchost – uchovává pouze aktuálně nejlepší nalezené řešení [14].

Algoritmus opakovaně generuje nová potenciální řešení, která vyhodnocuje pomocí fitness funkce. Pokud je nové řešení lepší než dosud nejlepší, nahradí ho. Algoritmus končí po dosažení maximálního počtu iterací nebo požadované kvality řešení [15].

Hlavní nevýhodou je velmi nízká efektivita – absence strategie vede k pomalé konvergenci a nutnosti velkého počtu iterací [15]. V praxi se proto používá spíše jako referenční metoda pro srovnání s pokročilejšími algoritmy [13].

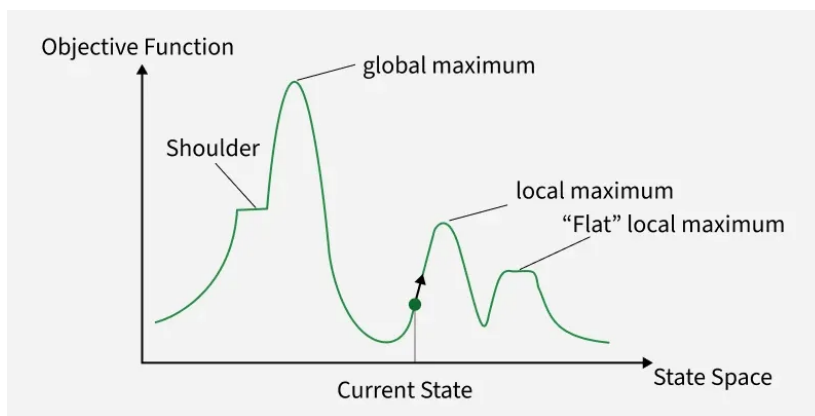
1.2.2 Horolezecký algoritmus

Horolezecký algoritmus (Hill Climbing) je lokální vyhledávací metoda, která systematicky prohledává stavový prostor ve směru zlepšování hodnoty cílové funkce [15]. Algoritmus začíná z náhodně zvoleného bodu a v každém kroku se přesouvá k sousednímu řešení s lepším ohodnocením, dokud nenajde lokální optimum [16].

V každé iteraci je generována množina sousedních řešení, z nichž je vybrán souseď s nejlepší hodnotou fitness funkce. Algoritmus pokračuje, dokud nenajde lokální optimum, ve kterém všechna sousední řešení mají horší ohodnocení. Existují různé varianty (simple, steepest ascent, stochastic), které se liší způsobem výběru sousedního řešení [1].

Zásadním problémem horolezeckého algoritmu je jeho neschopnost rozlišit mezi lokálním a globálním optím. Algoritmus vždy konverguje k nejbližšímu lokálnímu optím, které nemusí být nejlepším řešením v celém stavovém prostoru [16]. Tento problém ilustruje obrázek 1.1.

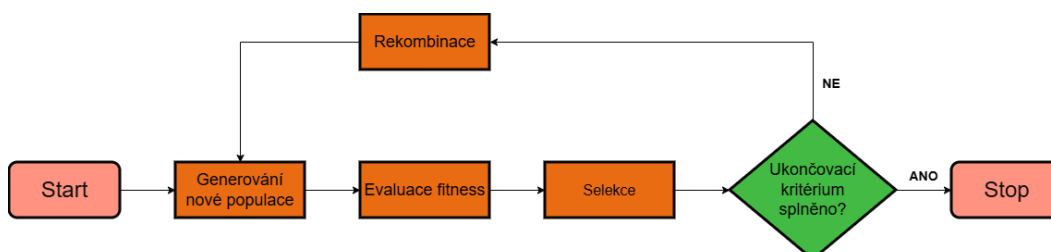
V případě symbolické regrese, kde prostor možných funkcí je vysoce nekonvexní s mnoha lokálními optímy, tento problém výrazně snižuje efektivitu metody [16].



Obrázek 1.1: Problém lokálního a globálního maxima při prohledávání stavového prostoru. Algoritmus může uvíznout v lokálním optimu. Zdroj: [1]

1.2.3 Evoluční algoritmy

Evoluční algoritmy (Evolutionary Algorithms, EA) představují pokročilou třídu optimalizačních metod, které kombinují výhody náhodného prohledávání se systematickým přístupem [16]. Na rozdíl od náhodného prohledávání, které pracuje s jediným řešením, a horolezeckého algoritmu, který uvízne v lokálním optimu, evoluční algoritmy pracují s populací řešení a dokáží efektivně prohledávat více oblastí stavového prostoru současně [15]. Na obrázku 1.2 je znázorněna smyčka průchodu algoritmem.



Obrázek 1.2: Diagram znázorňující chod evolučního algoritmu (inspirováno [2])

Tato schopnost souběžného zkoumání více kandidátních řešení umožňuje lépe prozkoumat stavový prostor než náhodné či lokální metody, což činí evoluční algoritmy vhodnými pro složité úlohy jako symbolická regrese. Podrobný popis evolučních algoritmů, jejich historie a fungování je uveden v následující kapitole 1.3.

1.3 Evoluční algoritmy

1.3.1 Historie a vznik

Evoluční algoritmy vznikly ze snahy využít principy biologické evoluce při řešení složitých optimalizačních úloh. Jejich vývoj byl ovlivněn třemi nezávislými výzkumnými

směry, které se formovaly v 60. a 70. letech 20. století: evolučními strategiemi (ES), evolučním programováním (EP) a genetickými algoritmy (GA).

Tři hlavní směry evolučních algoritmů vznikaly v 60. letech nezávisle na sobě. Evoluční strategie (ES) byly rozvíjeny od roku 1964 Ingem Rechenbergem na Technické univerzitě v Berlíně, kde řešil úlohy z mechaniky tekutin [17]. V roce 1966 představil Lawrence Fogel evoluční programování (EP), které kladlo větší důraz na modelování procesu evoluce pomocí konečných automatů [18]. Třetí směr tvoří genetické algoritmy (GA) Johna Hollanda z roku 1975, který představil binární reprezentaci řešení a jako klíčový prvek zavedl křížení [19].

Ačkoli ES, EP a GA používaly různé reprezentace, spojovala je myšlenka vycházející z Darwinovy teorie přirozeného výběru. Postupem času se tyto přístupy přiblížily a dnes jsou považovány za varianty evolučních algoritmů. Dalším významným milníkem bylo představení genetického programování Johnem Kozou v roce 1992, které je popsáno v kapitole 1.3.4 [15]. Od 90. let prošly evoluční algoritmy rychlým vývojem a nachází uplatnění v optimalizaci, strojovém učení, robotice, bioinformatice i dalších oborech [15, 16].

1.3.2 Základní principy

Jak bylo zmíněno, evoluční algoritmy jsou inspirovány přírodou, konkrétně Darwinovou teorií přirozeného výběru. Základním principem této teorie je, že v prostředí s omezenými zdroji přežívají jedinci, kteří jsou nejlépe přizpůsobení. V kontextu evolučních algoritmů to znamená, že kandidátní řešení mezi sebou soutěží o postup do další generace, přičemž ta kvalitnější mají vyšší šanci na přežití i reprodukci. Algoritmus tak opakovaně testuje a vybírá nejlepší řešení, dokud není splněna podmínka pro ukončení [16].

Evoluční algoritmus začíná vytvořením počáteční populace, která může být vygenerována zcela náhodně nebo může vycházet z již známých výsledků. V každé iteraci algoritmu, označované jako generace, algoritmus nejprve ohodnotí všechny jedince v populaci pomocí fitness funkce, která určuje kvalitu řešení. Na základě tohoto ohodnocení je vybrána množina rodičů, ze které se vytvoří předem daný počet potomků (*offspringů*) označovaný parametrem λ . Potomci vznikají aplikací genetických operátorů, jako je mutace nebo křížení [15].

Každý potomek je následně opět ohodnocen a algoritmus vybere jedince, kteří budou tvořit populaci rodičů v další generaci. Od správně navrženého algoritmu se očekává, že kvalita jedinců bude postupně růst, protože evoluční tlak (preferance jedinců s lepším fitness při selekci) posouvá populaci směrem k výhodnějším oblastem stavového prostoru [15]. Proces průchodu algoritmem je znázorněn ve schématu algoritmu 1.

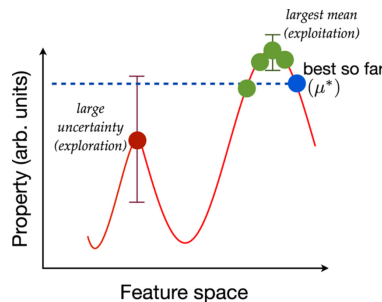
Algorithm 1 Evoluční algoritmus typu $(1+\lambda)$ – vlastní zpracování

```
1: Vytvoř počátečního rodiče  $p$ 
2: for každou generaci až do maximálního počtu do
3:   Ohodnot rodiče pomocí fitness funkce
4:   Vytvoř  $\lambda$  potomků aplikací genetických operátorů na rodiče
5:   Ohodnot všechny potomky
6:   Vyber nejlepšího jedince z množiny rodič + potomci
7:   Tento jedinec se stává rodičem pro další generaci
8:   if je dosaženo cílové fitness then
9:     break
10:  end if
11: end for
12: return nejlepší nalezený jedinec
```

Genetické operátory se typicky neaplikují při každém průchodu, nýbrž podle pravděpodobností stanovených uživatelem. Algoritmus probíhá, dokud není dosaženo maximálního počtu generací nebo nalezeno řešení požadované kvality [15].

Práce s populací více kandidátních řešení současně výrazně zvyšuje efektivitu algoritmu oproti metodám pracujícím s jediným řešením. Díky tomu může algoritmus prohledávat více oblastí stavového prostoru najednou. Genetické operátory, jako selekce nebo mutace, umožňují algoritmu uniknout z lokálních optim, protože do procesu vnášejí prvek náhodnosti. Algoritmus se tak může dostat i do dosud neprozkoumaných částí stavového prostoru [15].

Účinné evoluční algoritmy udržují rovnováhu mezi podrobným zkoumáním okolí nejlepšího řešení (exploatací) a hledáním nových oblastí stavového prostoru (explorací). Tento princip a vzájemný kompromis mezi oběma přístupy je ilustrován na obrázku 1.3. Tato schopnost je jednou z jejich největších výhod – například oproti horolezeckému algoritmu, který spoléhá pouze na exploataci. Kvalita evolučního algoritmu také úzce souvisí s vhodně navrženou fitness funkcí, která nesmí být ani příliš přísná (riziko předčasné konvergence), ani příliš volná (pomalé zlepšování) [15].



Obrázek 1.3: Schéma ilustrující kompromis mezi explorací a exploatací. Zdroj: [3]

1.3.3 Pojmy evolučních algoritmů

Populace a generace

Populace je množina jedinců, se kterými algoritmus pracuje v daném cyklu evoluce. Každá populace tvoří jednu generaci, která se na základě selekce, křížení a mutací postupně mění. Evoluční algoritmus tak iterativně vede populaci k lepším řešením.

Fitness

Fitness funkce je klíčovým prvkem, který ovlivňuje průběh evolučních algoritmů. Jedná se o funkci, která přiřazuje každému kandidátnímu řešení (jedinci) číselnou hodnotu vyjadřující jeho kvalitu vzhledem k řešenému problému. Většinou platí, že čím vyšší hodnota fitness, tím lepší je dané řešení. Ovšem v případech, kdy řešíme optimalizační problém s cílem minimalizace, je vhodnější použít inverzní přístup, kdy nižší hodnota znamená lepší řešení – v takovém případě se často hovoří o tzv. cost function [16].

Fitness funkce přímo ovlivňuje, která řešení budou vybrána pro reprodukci a která nikoliv. Díky tomu evoluční algoritmus směřuje populaci k lepším jedincům v průběhu generací. Volba vhodné fitness funkce je klíčová pro úspěch algoritmu – špatně navržená funkce může vést k předčasné konvergenci, nebo naopak k velmi pomalému zlepšování. Konkrétní podoba fitness funkce závisí na tom, jaký problém řešíme. Existuje několik typů fitness funkcí, například přizpůsobená fitness, hrubá fitness nebo normalizovaná fitness. V případě symbolické regrese se používají metriky měřící chybu mezi předpovězenými a skutečnými hodnotami, jako jsou MSE a MAE, které budou podrobně popsány v kapitole 1.4.1 [15].

Mutace

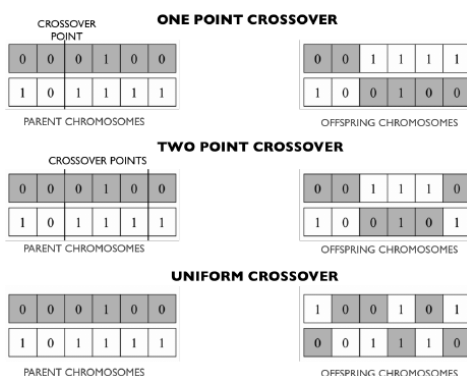
Mutace je genetický operátor, který se používá k zavedení náhodných změn do řešení. Pravděpodobnost aplikování mutace nastavuje uživatel a obvykle se nastavuje na velmi malou hodnotu (přibližně 0,1 %). Mutace musí být nastavena tak, aby po jejím aplikování nevznikaly neplatné hodnoty pro daný problém [15].

Mutace hraje odlišnou roli v různých implementacích evolučních algoritmů. Velkou roli má například v evolučních strategiích a evolučním programování, kde mutace využívá Gaussovo rozložení s nulovou střední hodnotou a rozptylem σ , který je upravován tak, aby byla přibližně pětina mutací úspěšná. Tento postup je znám jako „pravidlo jedné pětiny“ [15, 16].

Křížení

Křížení (crossover, recombination) je genetický operátor, který generuje potomky kombinací vlastností dvou rodičů. Jde o stochastický proces s typickou pravděpodobností provedení okolo 70 % [15, 16].

Křížení může být jednobodové (prohození části genetické informace za bodem křížení), vícebodové (prohození na více místech pro výraznější promíchání vlastností), nebo uniformní (prohazování jednotlivých genů podle určité pravděpodobnosti). Princip těchto typů křížení je znázorněn na obrázku 1.4 [15].



Obrázek 1.4: Schéma ilustrující způsoby křížení. Zdroj: [4]

Míra využití tohoto operátoru se liší v jednotlivých implementacích EA. V genetických algoritmech (GA) je křížení klíčovým mechanismem, zatímco v evolučním programování (EP) se běžně nepoužívá [16].

Selekce

Selekce je genetický operátor, který zajišťuje výběr jedinců pokračujících v evoluci. Je důležité navrhnout selekční mechanismus vyváženě – dostatečně upřednostňovat jedince s vyšším fitness, ale zároveň zachovat různorodost populace, aby se algoritmus nezasekl v lokálním optimu [15].

Rankingová selekce snižuje riziko předčasné konvergence tím, že jedinci jsou vybíráni podle pozice v seřazené populaci, nikoli podle absolutní fitness hodnoty. Každá pozice má přiřazenu určitou pravděpodobnost výběru, což zabraňuje dominanci jedinců s extrémně vysokou fitness [15, 16].

Turnajová selekce náhodně vybírá jedince do turnajů, kde mezi sebou soupeří na základě relativní fitness. Vítězem se stává jedinec s nejvyšší hodnotou fitness. Proces se opakuje, dokud není vybrán požadovaný počet jedinců. Tato metoda je oblíbená díky jednoduchosti a dobře kontrolovatelné selekční intenzitě [15, 16].

Elitismus zaručuje, že nejlepší jedinci (obvykle 1–5 % populace) postupují do další generace beze změny. Aplikuje se před působením genetických operátorů, aby nedošlo ke znehodnocení jejich kvalit [16].

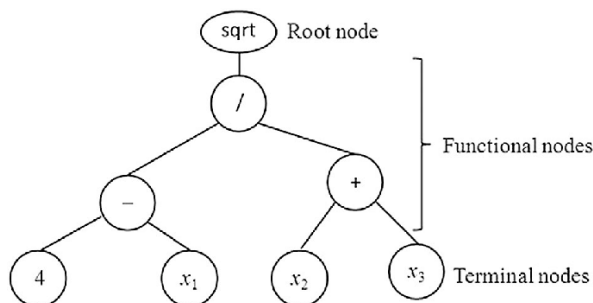
Ruletová selekce přiřazuje pravděpodobnost výběru úměrnou fitness jedince ($P_i = f_i / \sum_{j=1}^n f_j$), čímž zvýhodňuje silnější jedince při zachování šance i pro slabší [15, 16].

1.3.4 Genetické programování

Genetické programování (GP) je konkrétní implementace evolučních algoritmů. Tato metoda byla poprvé představena v roce 1992 v knize *Genetic Programming: On the Programming of Computers by Means of Natural Selection* autorem Johnem Kozou [20], který je díky tomu považován za zakladatele tohoto odvětví.

GP umožňuje řešit složité úlohy bez potřeby explicitního definování vztahů či struktur, které mají být hledány. Z genetických operátorů využívá hlavně mutaci a křížení. Pro reprezentaci problémů GP typicky používá stromovou strukturu, kde konstanty (např. x , y , 3, ...) jsou nazývány terminály a funkce (+, -, $\times$, $\div$, sin, cos, exp, ...) tvoří funkční uzly [21]. Stromovou reprezentaci lze vidět na obrázku 1.5.

Specifickou variantou genetického programování je kartézské genetické programování (CGP), které bylo použito v mém programu a je podrobněji popsáno v další podkapitole.



Obrázek 1.5: Schéma ilustrující stromovou strukturu. Zdroj: [5]

Kartézské genetické programování

Kartézské genetické programování (CGP) je metoda GP, která byla vytvořena Julianem F. Millerem na konci 90. let. Přídavné jméno „kartézské“ vychází z dvourozměrné reprezentace mřížky uzlů, kterou CGP používá. Kandidátní řešení jsou reprezentována orientovanými grafy, čímž se CGP liší od klasického GP, kde jsou reprezentována pomocí stromů [15, 22].

Hlavní rozdíl mezi stromovou a grafovou reprezentací spočívá ve způsobu využití uzlů. V případě stromové struktury je každý uzel použit právě jednou. Naproti tomu grafová

reprezentace umožňuje, aby některé uzly byly použity opakovaně, zatímco jiné uzly mohou zůstat neaktivní – tyto uzly existují v grafu, ale nejsou zapojeny do výpočtu výsledku. Z toho plyne, že zatímco u stromové reprezentace délka stromu přímo určuje složitost výsledného výrazu, u grafové reprezentace může být délka výsledného výrazu kdekoliv v intervalu od nuly až po počet uzlů daného jedince [22].

Struktura CGP je založena na dvourozměrné mřížce o rozměrech $n_r \times n_c$, kde n_r je počet řádků a n_c počet sloupců. Velikost této mřížky je pevně stanovena před začátkem evoluce. Každý prvek mřížky obsahuje funkční uzel se zadaným počtem vstupů (závislých na aritě funkce) a jedním výstupem. Vstupní geny v i -tém sloupci mohou být připojeny pouze k vstupním uzlům (proměnným nebo konstantám), nebo k výstupům uzlů z předchozích sloupců. Uzly nemohou být připojeny k uzlům ze stejného sloupce ani z následujících sloupců, čímž je zaručeno, že výsledný graf bude acyklický. Strategie prohledávání u CGP je označována jako ES($1 + \lambda$), kde jeden rodič vytváří λ potomků a do další generace postupuje nejlepší z nich [22].

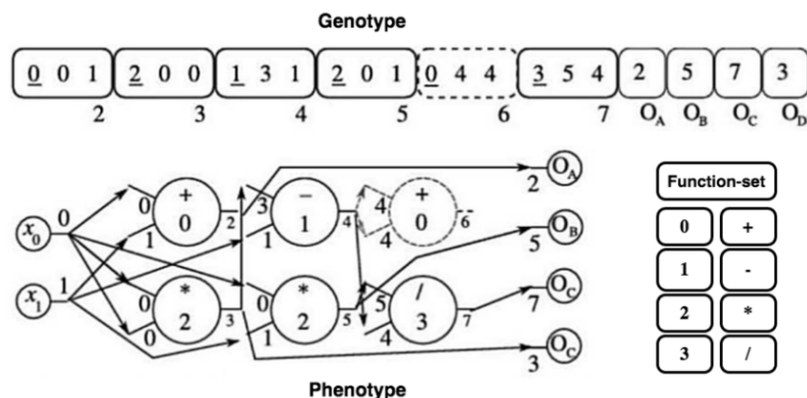
Rozsah připojení vstupů uzlů je omezen parametrem levels-back (L), který určuje maximální počet předchozích sloupců, ze kterých může uzel čerpat vstupy. [15].

Genotyp CGP je reprezentován jako posloupnost celých čísel, která kóduje strukturu celé mřížky. Pro každý uzel jsou v genotypu uloženy dvě informace: identifikátor funkce (například 0 = sčítání, 1 = násobení, 2 = sinus) a indexy uzlů, ze kterých uzel přijímá data. Indexy jsou číslovány od 0 a mohou odkazovat buď na vstupy programu (proměnné a konstanty), nebo na výstupy předchozích uzlů v mřížce podle parametru levels-back. Identifikátory funkcí jsou reprezentovány indexem podle pozice funkce v množině povolených funkcí [15, 22].

Genotyp má fixní délku, která je určena velikostí mřížky a počtem výstupů. Příklad genotypu pro uzel s dvěma vstupy a funkcí sčítání by mohl vypadat následovně: [0, 1, 2], kde číslo 0 označuje index funkce z množiny funkcí a čísla 1 a 2 označují vstupy uzlu. Celý genotyp pak obsahuje tuto informaci pro všechny uzly v mřížce následovaný indexy výstupních uzlů [22].

Fenotyp je výsledná posloupnost a zahrnuje pouze aktivní uzly, tedy ty, které se podílejí na výpočtu výstupu programu. Některé uzly zakódované v genotypu nemusí být využity vůbec, zatímco jiné mohou být naopak používány opakovaně, pokud na jejich výstup odkazuje více následných uzlů [15]. Rozdíl mezi fenotypem a genotypem je znázorněn na obrázku 1.6.

Z genetických operátorů využívá CGP především mutaci (některé varianty sice umožňují i použití křížení, avšak to se v praxi objevuje pouze zřídka). Mutace v CGP probíhá jako jednobodová – algoritmus náhodně vybere gen, u kterého provede změnu. Mutovat je možné jak funkční gen (identifikátor funkce), tak vstupní gen (index uzlu, ze kterého



Obrázek 1.6: Schéma reprezentace CGP: mřížka uzlů s genotypem (posloupnost celých čísel) a fenotypem (aktivní uzly tvořící výsledný program) Zdroj: [6]

uzel přijímá vstup). Počet genů, které algoritmus bude mutovat v jedné generaci, je obvykle vyjádřen jako podíl z celkového množství. Při mutaci je nutné zachovat validitu vzniklého genotypu vzhledem ke struktuře mřížky, tedy aby nově vzniklé indexy odkazovaly pouze na povolené vstupy nebo uzly z předchozích sloupců. Například při mutaci vstupního genu uzlu v 7. sloupci nemůže vzniknout odkaz na uzel v 8. sloupci. Platné hodnoty tedy odpovídají pouze indexům vstupů a uzlů z předchozích sloupců (případně konstantám). V CGP také platí pravidlo, že pokud potomek má stejnou hodnotu fitness jako jeho rodič, tak z důvodu diverzity, bereme do další generace potomka [15, 22].

Hlavní výhodou kartézského genetického programování je jeho jednoduchá a kompaktní reprezentace, která umožňuje efektivní evoluci i u velmi složitých úloh. Díky grafové struktuře mohou být některé uzly znovu využívány, což často vede k menším a lépe generalizujícím řešením. Důsledkem reprezentace genů pomocí čísel je, že CGP je také jednoduché na implementaci a nemá vysoké nároky na výkon.

1.4 Symbolická regrese

Symbolická regrese (SR) je metoda strojového učení, jejímž cílem je automatické objevování matematických vztahů mezi vstupními proměnnými a výstupní veličinou. Na rozdíl od klasických regresních metod, jako je lineární nebo polynomická regrese, nevyžaduje symbolická regrese předem definovanou formu matematického výrazu, kterou má hledat. Místo toho sama prohledává prostor možných matematických výrazů a snaží se najít ten, který nejlépe popisuje naměřená data [11, 21].

Historie symbolické regrese sahá do 70. let 20. století, kdy Patrick Langley vyvinul systém BACON, který byl schopen objevovat jednoduché fyzikální zákony z experimentálních dat. Zásadní průlom však nastal až v 90. letech, kdy John Koza propojil symbolickou regresi s genetickým programováním (GP). Koza ukázal, že evoluční al-

goritmy inspirované biologickou evolucí dokážou efektivně hledat vhodné matematické modely [23].

Klíčovou výhodou symbolické regrese oproti klasickým metodám je schopnost vyvažovat přesnost predikce se složitostí výsledného výrazu. Zatímco složité modely mohou dosahovat nižší chyby na trénovacích datech, jednodušší výrazy, které vytváří symbolická regrese, jsou často lépe generalizovány na nová data a jsou snadněji interpretovatelné. Symbolická regrese tedy nehledá pouze nejpřesnější vztah, ale optimální kompromis mezi přesností a složitostí výrazu, což je klíčové pro praktické aplikace v přírodních vědách a inženýrství [23].

V následujících podkapitolách jsou představeny metriky pro vyhodnocování modelů, hlavní výzvy této metody a její konkrétní praktické využití.

1.4.1 Metriky vyhodnocování v SR

V symbolické regresi existuje celá řada metrik pro vyhodnocování kvality nalezeného modelu. V této podkapitole budou představeny ty nejdůležitější, které se používají jak při trénování, tak při porovnávání výsledných výrazů. Výběr optimální metriky je závislý na charakteru problému, běžně se používá kombinace těchto metrik [24].

MSE

Střední kvadratická chyba (*mean squared error*, MSE) je jedna z nejpoužívanějších metrik pro hodnocení kvality regresních modelů. Vyjadřuje průměrnou velikost kvadratických odchylek mezi skutečnými hodnotami a hodnotami predikovanými modelem. Výhodou MSE je skutečnost, že kvadratické umocnění výrazně penalizuje větší chyby, čímž nutí model řešit zejména výrazné odchylky od skutečných hodnot. MSE se počítá následujícím vztahem [25]:

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y[i] - \hat{y}[i])^2,$$

kde N je počet vzorků, $y[i]$ skutečná hodnota a $\hat{y}[i]$ hodnota predikovaná modelem.

RMSE

RMSE (*root mean squared error*) je odvozená metrika ze střední kvadratické chyby MSE. Vzniká aplikací druhé odmocniny na hodnotu MSE, čímž se výsledek převádí zpět do stejných jednotek, v jakých jsou původní data (například chyba v centimetrech místo cm^2) [25].

Důležité je, že odmocnina se počítá až z výsledné hodnoty MSE, nikoli z jednotlivých rozdílů před jejich zprůměrováním. Vztah pro výpočet RMSE je následující [25]:

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N (y[i] - \hat{y}[i])^2}$$

MAE

Střední absolutní chyba (*mean absolute error*, *MAE*) je metrika, která se na rozdíl od předchozích nepočítá s kvadratickou chybou, ale součtem absolutních rozdílů mezi skutečnými a predikovanými hodnotami. MAE tedy nepenalizuje velké odchylky výrazně více než malé a stejně jako RMSE zachovává původní jednotky měřených veličin. Vztah pro výpočet MAE je následující [25]:

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y[i] - \hat{y}[i]|$$

R^2

Koeficient determinace (*coefficient of determination*, R^2) je metrika určující, jakou část variability skutečných hodnot dokáže model vysvětlit. Hodnota R^2 se obvykle pohybuje v intervalu $(-\infty, 1]$, přičemž hodnota 1 značí perfektní přizpůsobení, hodnota 0 znamená, že model nevysvětluje žádnou variabilitu dat, a záporné hodnoty indikují, že model je horší než predikce průměru [24]. Například hodnota $R^2 = 0,6$ znamená, že model vysvětluje 60 % variability dat.

$$R^2 = 1 - \frac{\sum_{i=1}^N (y[i] - \hat{y}[i])^2}{\sum_{i=1}^N (y[i] - \bar{y})^2},$$

kde $\bar{y}$ je průměr skutečných hodnot.

1.4.2 Problémy a výzvy SR

Jedním z hlavních problémů symbolické regrese je tzv. *bloat*, tedy nekontrolovaný růst délky a strukturální složitosti výsledných matematických výrazů bez odpovídajícího zlepšení kvality řešení. Modely mohou obsahovat nadbytečné části, které mají minimální nebo žádný vliv na výslednou hodnotu fitness. Bloat negativně ovlivňuje nejen interpretovatelnost nalezených modelů, ale také zvyšuje jejich výpočetní náročnost a snižuje schopnost generalizace na dosud neviděná data. V praxi je proto často kladen důraz na metody, které podporují jednodušší struktury, například pomocí penalizace složitosti ve fitness funkci nebo následné symbolické simplifikace [26].

Dalším významným problémem symbolické regrese je tzv. *overfitting*, tedy situace, kdy nalezený matematický vztah velmi přesně odpovídá trénovacím datům, avšak postrádá schopnost zobecnění na nová data. Tento jev se často vyskytuje zejména u datasetů

obsahujících šum, kde algoritmus může nalézt extrémně složitý výraz, který přesně prochází všemi trénovacími body, místo aby vystihl obecný trend dat. Model tak fakticky popisuje náhodné odchylky způsobené šumem, nikoli skutečný vztah mezi proměnnými [26].

Uvedené problémy významně ovlivňují návrh fitness funkcí, volbu evolučních operátorů i nastavení parametrů algoritmu, což je zohledněno v praktické části této práce.

1.4.3 Aplikace SR

Symbolická regrese se využívá v mnoha vědních oborech, kde je důležitá jednoduchost a interpretovatelnost vztahu mezi proměnnými. Pomáhá například při objevování nových fyzikálních zákonů nebo při upřesňování již známých, kdy dokáže hledat jednodušší výrazy ze složitějších modelů [26].

Ve zdravotnictví nachází symbolická regrese využití například při identifikaci klíčových atributů pro predikci přežití pacientů po srdečním selhání nebo při analýze vlivu různých faktorů na výskyt nemocí [26]. Umožňuje tedy lékařům lépe porozumět souvislostem v datech a podporuje rozhodování založené na důkazech.

Kromě fyziky a medicíny nachází symbolická regrese uplatnění i v oblasti financí, například při modelování cen finančních aktiv a objevování vztahů mezi tržními faktory. Tento typ použití umožňuje vytvářet interpretovatelné matematické modely, které zachycují složité ekonomické vztahy přímo z dat [27].

2 Cíle práce a návrh

2.1 Cíle práce

Hlavním cílem této práce je vytvořit funkční interaktivní webovou aplikaci pro symbolickou regresi, která umožní uživateli experimentovat s parametry evolučního algoritmu a sledovat průběh hledání matematických vztahů v reálném čase. Aplikace má sloužit jako vzdělávací i experimentální nástroj pro pochopení principů kartézského genetického programování a jeho aplikace v symbolické regresi.

Dílčí cíle práce jsou následující:

1. **Implementovat evoluční algoritmus založený na CGP**

Vytvořit funkční implementaci kartézského genetického programování (CGP), která bude schopna evolučně hledat matematické výrazy popisující zadaná data.

2. **Navrhnout fitness funkci s penalizací**

Implementovat fitness funkci založenou na střední kvadratické chybě (MSE) s penalizačními mechanismy.

3. **Vytvořit intuitivní grafické uživatelské rozhraní**

Navrhnout a implementovat GUI, které umožní uživateli nastavovat parametry evoluce, načítat různé datasety a sledovat průběh evoluce prostřednictvím grafu a logů.

4. **Implementovat a porovnat různé režimy algoritmu**

Vytvořit tři strategie inicializace chromozomu (náhodná, s maximální délkou, výběr nejlepšího z N kandidátů) a dva režimy práce s konstantami (fixní vs. evolvovatelné), které umožní experimentální vyhodnocení jejich vlivu na kvalitu konvergence.

5. **Porovnat vliv parametrů a otestovat algoritmus**

Provést experimenty na datasetech reprezentujících různé typy závislostí (lineární, kvadratické, logaritmické, sinusové, komplexní), vyhodnotit a zdůraznit vliv parametrů na konvergenci. Ověřit, jak je CGP schopné pracovat s daty zatíženými šumem a zda dokáže z dat zrekonstruovat reálný fyzikální problém.

2.2 Návrh a volba technologií

2.2.1 Návrh evolučního algoritmu

Pro implementaci symbolické regrese bude zvolen přístup kartézského genetického programování (CGP), jehož principy byly představeny v kapitole 1.3.4. Volba CGP vychází z následujících praktických důvodů:

- Jednoduchá reprezentace pomocí pole celých čísel
- Možnost opakovaného využití funkčních uzlů ve fenotypu
- Nízká výpočetní náročnost

Algoritmus bude využívat evoluční strategii $(1+\lambda)$, kde jeden rodič vytváří λ potomků pomocí mutace. Všichni potomci jsou následně ohodnoceni fitness funkcí a nejlepší jedinec z množiny {rodič, potomci} postupuje do další generace. Tato strategie zaručí neklesající hodnotu fitness a zároveň, díky práci s pouze jedním rodičem, má nízkou výpočetní náročnost. Hlavním genetickým operátorem bude mutace. Křížení (crossover) nebude využito, což odpovídá standardnímu přístupu u CGP podle literatury [15, 22].

Chromozom bude reprezentován jako jednorozměrné pole celých čísel obsahující m trojic čísel reprezentujících funkční uzly, index výstupního uzlu a volitelně hodnoty evolvovatelných konstant. Každý funkční uzel se bude skládat ze tří čísel – první dvě budou reprezentovat vstupy uzlu a třetí identifikátor funkce.

Aplikace umožní uživateli nastavovat parametry evoluce prostřednictvím ovládacího panelu. Výchozí hodnoty parametrů jsou shrnuty v tabulce 2.1. Uživatel bude moci také vybírat ze tří strategií inicializace počáteční populace a volit mezi fixními a evolvujícími konstantami.

Parametr	Výchozí hodnota
Počet potomků (λ)	20
Počet generací	5000
Maximální počet uzlů	15
Pravděpodobnost mutace	0.2
Strategie inicializace	náhodná
Konstanty	evolvovatelné (3)

Tabulka 2.1: Výchozí parametry evolučního algoritmu

2.2.2 Návrh fitness funkce

Fitness funkce bude založena na střední kvadratické chybě (MSE), která byla popsána v kapitole 1.4.1. Důvod, proč jsem zvolil tuto metriku, je, že silně penalizuje velmi špatná řešení, což pomáhá k rychlejší konvergenci, a zároveň její nevýhoda – umocňování jednotek na druhou – v mém případě nevádí, neboť hodnoty jsou bezjednotkové. Pro lepší interpretovatelnost bude fitness definována jako negace MSE, aby vyšší hodnota fitness odpovídala lepšímu řešení:

$$\text{fitness} = -\text{MSE} = -\frac{1}{N} \sum_{i=1}^N (y[i] - \hat{y}[i])^2 \quad (2.1)$$

Fitness funkce bude dále rozšířena o penalizaci složitosti výrazu. Tento mechanismus slouží k předcházení nadměrnému růstu výrazu (*bloat*) a snižování rizika přeučení (*overfitting*), které byly popsány v kapitole 1.4.2.

2.2.3 Návrh grafického rozhraní

Aplikace bude implementována jako webová stránka. Struktura rozhraní bude rozdělena na dvě části: levý panel s ovládacími prvky a hlavní panel pro vizualizaci a výsledky. Návrh rozhraní je znázorněn na obrázku 2.1.

Interaktivní symbolická regrese s využitím AI		
Panel nastavení	Graf	Tlačítko pro spuštění
Výběr datasetu:		Log evoluce
Povolené funkce		
Způsob inicializace:		
Typ a počet konstant	Výsledný matematický výraz	Reprezentace chromozomem
Parametry evoluce	Tabulka porovnávající vypočtené a reálné hodnoty y	

Obrázek 2.1: Wireframe grafického rozhraní aplikace

Levý panel bude obsahovat ovládací prvky pro výběr datasetu, povolených funkcí, typu inicializace, typu konstant a parametrů evoluce. Hlavní oblast bude obsahovat graf zobrazující data a nalezenou funkci, tlačítka pro ovládání evoluce, log průběhu, výsledný předpis v LaTeX notaci a tabulku porovnání hodnot.

2.2.4 Volba technologií

Pro implementaci budou zvoleny následující technologie, přičemž ohled bude brán na požadavky projektu – interaktivitu, dostupnost v prohlížeči a snadnou údržbu.

Frontend technologie

- **Vue.js 3** bude zvolen jako framework pro uživatelské rozhraní díky vestavěné reaktivitě a jednoduché syntaxi oproti alternativám jako *React* nebo *Angular*. Vue umožňuje automatickou aktualizaci zobrazení při změně dat, což je klíčové pro vizualizaci průběhu evoluce v reálném čase. Nízká vstupní bariéra usnadňuje další rozšíření projektu.
- **Chart.js** bude preferován před komplexnější knihovnou *D3.js* pro svou malou velikost (přibližně 40 kB minifikovaný) a jednoduché API. Pro účely zobrazení scatter a line grafů je Chart.js plně dostačující, zatímco D3.js (přibližně 200 kB) by přinášelo zbytečnou složitost.
- **MathJax 3** – standardní řešení pro vykreslování matematických výrazů v *LaTeX* notaci přímo v prohlížeči. Umožňuje zobrazit výsledný funkční předpis v profesionální typografické kvalitě bez nutnosti konverze do jiného formátu.

Vývojové nástroje

- **Node.js + npm** – standardní ekosystém pro správu JavaScriptových závislostí a build proces. Umožňuje snadnou instalaci knihoven a automatizaci vývojových procesů.
- **Vite** – moderní build nástroj, který nahrazuje starší *webpack*. Bude zvolen pro rychlý development server s hot module replacement a optimalizovaný produkční build. Vite je výrazně rychlejší než tradiční webpack-based řešení.
- **Git + GitHub** – verzování kódu pomocí Gitu zajistí sledování změn a možnost návratu k předchozím verzím. GitHub bude sloužit jako veřejné úložiště projektu a umožní automatický deployment prostřednictvím GitHub Actions.

Kombinace těchto technologií umožní vytvoření jednostránkové webové aplikace (SPA), která bude běžet přímo v prohlížeči bez nutnosti instalace. Aplikace bude nasazena na GitHub Pages a při každé změně v repozitáři se automaticky aktualizuje, což zajistí její trvalou online dostupnost pro uživatele.

3 Implementace webové aplikace

Tato kapitola popisuje implementaci webové aplikace pro symbolickou regresi založenou na kartézském genetickém programování. Aplikace byla navržena jako interaktivní nástroj umožňující experimentovat s parametry evolučního algoritmu a sledovat průběh evoluce v reálném čase.

Implementace je rozdělena do dvou hlavních částí:

- **Evoluční algoritmus** představuje výpočetní jádro – implementuje principy CGP z kapitoly 1.3.4 a zajišťuje hledání optimálního matematického výrazu.
- **Grafické uživatelské rozhraní** zprostředkovává komunikaci s uživatelem, umožňuje nastavení parametrů a vizualizuje výsledky evoluce.

Aplikace je realizována jako jednostránková webová aplikace (SPA) běžící v prohlížeči. Tato architektura umožňuje okamžitou dostupnost bez instalace a funguje na všech operačních systémech. Vue.js zajišťuje reaktivní uživatelské rozhraní, kdy změny dat se automaticky promítají do zobrazení. JavaScript ES6+ s asynchronními funkcemi umožňuje provádět evoluční výpočty bez blokování rozhraní. Chart.js slouží k vizualizaci dat, MathJax 3 vykresluje matematické výrazy v LaTeX notaci a CSS Grid definuje strukturu rozhraní.

Datový tok je jednosměrný – uživatel nastavuje parametry uložené v reaktivních proměnných, evoluční algoritmus je čte a průběžně aktualizuje další reaktivní proměnné, které se automaticky zobrazují v rozhraní.

Pro vývoj aplikace byl použit build nástroj Vite, který zajišťuje rychlý development server (`npm run dev`) pro testování během vývoje a optimalizovaný produkční build (`npm run build`). S pomocí externího konzultanta byl nastaven GitHub Actions workflow pro automatické publikování aplikace na GitHub Pages při každém commitu do hlavní větve. Aplikace je tak trvale dostupná online bez nutnosti lokální instalace, což usnadňuje její použití.

V následujících podkapitolách je nejprve podrobně popsána implementace evolučního algoritmu a poté grafického uživatelského rozhraní. Kompletní zdrojový kód aplikace je veřejně dostupný na platformě GitHub.

3.1 Evoluční algoritmus

3.1.1 Hlavní evoluční smyčka

Funkce `EvolucniAlgoritmus()` představuje jádro celého programu. Tato funkce je zodpovědná za průběh evoluce a při jejím průchodu se volají další pomocné funkce popsané v následujících podkapitolách.

Funkce implementuje evoluční strategii $(1+\lambda)$. Evoluce začíná vytvořením rodiče (parent), pro kterého je následně vygenerováno λ potomků (offspring) pomocí mutace, přičemž hodnota λ je nastavitelná uživatelem. Tito potomci jsou ohodnoceni fitness funkcí a z množiny {rodič, potomci} je vybrán jedinec s nejvyšší hodnotou fitness. Pokud má potomek stejnou fitness jako rodič, je z důvodu zachování diverzity vybrán potomek. Tato strategie byla zvolena pro svou jednoduchost a záruku neklesající fitness hodnoty. Evoluční smyčka je znázorněna v následujícím algoritmu 2:

Algorithm 2 Evoluční algoritmus $(1+\lambda)$

```
1: Inicializuj rodiče  $p$ 
2: for  $g = 1$  to maxGenerací do
3:   bestOffspring  $\leftarrow p$ 
4:   for  $i = 1$  to  $\lambda$  do
5:     offspring  $\leftarrow$  Mutace( $p$ )
6:     if Fitness(offspring)  $\geq$  Fitness(bestOffspring) then
7:       bestOffspring  $\leftarrow$  offspring
8:     end if
9:   end for
10:   $p \leftarrow$  bestOffspring
11: end for
12: return  $p$ 
```

Funkce je implementována jako asynchronní s pravidelnými přerušeními výpočtu, což umožňuje prohlížeči zpracovávat uživatelské vstupy a aktualizovat zobrazení. Aktualizace grafu probíhá každých 5 generací a zápis do logu každých 10 generací, čímž zůstává grafické rozhraní responzivní i při dlouhotrvajících evolucích.

Evoluční smyčka může být ukončena 3 způsoby – dosažením maximálního počtu generací, manuálním zastavením uživatelem, nebo dosažením cílové hodnoty fitness (fitness $> -0,01$).

3.1.2 Reprezentace chromozomu

Chromozom představuje postup počítání hodnoty zakódovaný v číslech. Jeho reprezentací vzniká pro algoritmus interpretovatelný zápis, jak vypočítat výstupní hodnotu.

Chromozom byl reprezentován jednorozměrným polem uloženým v reaktivní proměnné **chromozom**. Skládá se ze tří částí: **Funkční uzly**, **Výstupní index** a **Evolvovatelné konstanty** (volitelné). Struktura chromozomu s n uzly je následující:

$$[\text{in1}[0], \text{in2}[0], \text{fn}, \text{in1}[1], \text{in2}[1], \text{fn}, \dots, \text{in1}[n], \text{in2}[n], \text{fn}, \text{output}, \text{ev}[1], \text{ev}[2], \dots, \text{ev}[k]]$$

Každý funkční uzel je reprezentován třemi geny – dva vstupy (**in1**, **in2**) a index funkce (**fn**), který odkazuje na pozici v poli **vybraneFunkce**. Tato fixní velikost je uložena v konstantě **VelikostChromozomu = 3**.

Indexování bylo navrženo následovně: Index 0 odkazuje na vstupní hodnotu x , indexy 1 až n na výstupy předchozích uzlů (každý uzel se může připojit na libovolný předchozí uzel), indexy 1001+ na fixní konstanty a indexy 2000+ na evolvovatelné konstanty. Toto rozdělení umožňuje rozlišit typy vstupů a zabraňuje kolizím.

Velikost chromozomu je určena proměnnou **COLS** (počet uzlů) a počtem evolvovatelných konstant k . Délka chromozomu je dána vztahem $\text{COLS} \times 3 + 1 + k$. Tato reprezentace umožňuje efektivní práci s chromozomem při evaluaci, která je podrobně rozebrána v kapitole 3.1.5.

3.1.3 Inicializace chromozomu

Inicializace chromozomu je způsob, jakým se vytváří počáteční rodič. V programu byly implementovány 3 způsoby inicializace, mezi kterými si může uživatel vybírat.

- **Náhodná inicializace**

Funkce **GenerovaniNahodnehoChromozomu()** vytváří chromozom čistě náhodně podle pravidel pro tvorbu chromozomu. Hodnota **in1** prvního uzlu je pevně nastavena na vstupní hodnotu x pro zaručení jejího použití. Výstupní index je nastaven na poslední uzel. Evolvovatelné konstanty jsou inicializovány pomocí Gaussova rozdělení.

- **Inicializace s maximální délkou**

Funkce **GenerovaniMaximalnihoChromozomu()** se liší od náhodné inicializace tím, že v každém uzlu **in1** odkazuje na bezprostředně předchozí uzel, což zaručuje propojení všech uzlů. Výstupní index je nastaven na poslední uzel, čímž je zajištěno, že všechny uzly budou aktivní a výsledný funkční předpis bude mít maximální délku.

- **Inicializace *Best of N***

Funkce `GenerovaniNejlepsihoChromozomu()` vygeneruje N náhodných chromozomů (výchozí hodnota $N = 100$), které následně ohodnotí fitness funkcí a jako rodiče vybere toho s nejvyšší hodnotou fitness. Tato metoda zaručuje kvalitnější počáteční řešení, avšak za cenu vyšší výpočetní náročnosti.

Switch funkce `GenerovaniChromozomu()` volá příslušnou inicializační funkci podle hodnoty uložené v reaktivní proměnné `typInicializace`, kterou nastavuje uživatel prostřednictvím grafického rozhraní.

3.1.4 Fitness funkce

Fitness funkce určuje kvalitu řešení, jinými slovy, jak přesně funkce popisuje data. Kvalitní fitness funkce je základem pro správnou selekci kandidátních řešení.

V programu byla podle návrhu implementována fitness funkce založená na střední kvadratické chybě (MSE). MSE byla zvolena pro její schopnost silně penalizovat velmi špatná řešení. Pro jednoduchou interpretaci byla fitness funkce implementována jako negace MSE, aby vyšší fitness hodnota reprezentovala kvalitnější řešení. Vzorec fitness funkce je následující:

$$\text{fitness} = -\text{MSE} = -\frac{1}{N} \sum_{i=1}^N (y[i] - \hat{y}[i])^2 \quad (3.1)$$

Pro správnou konvergenci byly v programu implementovány 3 penalizační mechanismy:

- **Penalizace neplatných hodnot**

Za každou neplatnou hodnotu ($-\infty$, NaN), kterou vrátí funkce `chrom_evaluate()`, se k MSE přičte 1000 a pokud těchto neplatných hodnot bude více než 50 %, je fitness okamžitě nastavena na $-\infty$. Tento mechanismus je zodpovědný za eliminaci nefunkčních chromozomů.

- **Penalizace nízké variability**

Pokud má cílový dataset dostatečnou variabilitu ($\sigma_{\text{dataset}}^2 > 0,01$), mechanismus kontroluje, zda predikce chromozomu nejsou téměř konstantní. Spočítá průměr validních predikovaných hodnot a pomocí vzorce $\sigma^2 = \frac{1}{N} \sum_{i=1}^N (\hat{y}[i] - \bar{y})^2$ vypočítá rozptyl. Je-li $\sigma^2 < 0,0001$, je fitness nastavena na $-\infty$, čímž jsou filtrovány degenerované chromozomy vracející konstantní hodnoty.

- **Penalizace za složitost (bloat prevence)**

Za každý aktivní funkční uzel je k MSE přičtena hodnota 0,002. Tato hodnota byla zvolena experimentálně jako optimální pro dostatečné preferování jednodušších výrazů. Počet aktivních uzlů je spočítán funkcí `SpocitejAktivniUzly()`.

Tento mechanismus nutí algoritmus upřednostňovat jednodušší výrazy a zabráňuje nadměrnému růstu délky výrazu.

Finální hodnota fitness funkce je tedy $-(\text{MSE} + \text{penalizace})$. Fitness funkce jako kombinace MSE a penalizačních mechanismů byla implementována za účelem zajištění funkčního a jednoduchého řešení.

3.1.5 Evaluace chromozomu

Evaluace chromozomu je proces, při kterém se pro zadaný vstup x vypočítá výstupní hodnota $\hat{y}$ podle zakódovaného předpisu v chromozomu. Tento proces je implementován funkcí `chrom_evaluate(chrom, valX)`, která je nejčastěji volanou funkcí v celém algoritmu, protože je volána při každém vyhodnocení fitness.

Funkce sekvenčně prochází všechny uzly chromozomu a do pole `values[]` ukládá mezivýsledky. Na začátku je `values[0] = valX`, což odpovídá vstupní hodnotě. Dále jsou inicializovány hodnoty konstant – v případě fixních konstant se hodnoty načtou z množiny vygenerované funkcí `VygenerujFixniKonstanty()` a uloží na indexy 1001+, v případě evolvovatelných konstant se hodnoty načtou přímo z chromozomu (offset = `COLS × 3 + 1`) a uloží na indexy 2000+. Výsledek i -tého uzlu je následně uložen do `values[i+1]`. Toto indexování bylo zvoleno z důvodu, aby hodnota nultého uzlu nebyla uložena do `values[0]`, kde je již vstupní proměnná x .

Příklad evaluace: Pro chromozom `[0, 1001, 0, 1, 1002, 1, 2, 1, 2, 3]`, fixní konstanty `values[1001] = -3` a `values[1002] = -2`, a vstup $x = 2$ probíhá evaluace následovně:

Uzel	in1	in2	Funkce	Výpočet	values[i]
0	<code>values[0]=2</code>	<code>values[1001]=-3</code>	+	$2 + (-3)$	<code>values[1]=-1</code>
1	<code>values[1]=-1</code>	<code>values[1002]=-2</code>	-	$-1 - (-2)$	<code>values[2]=1</code>
2	<code>values[2]=1</code>	<code>values[1]=-1</code>	$\times$	$1 \times (-1)$	<code>values[3]=-1</code>

Výstupní index je 3, tedy výsledek je `values[3] = -1`. Tento chromozom tedy reprezentuje funkci $f(x) = (x + (-3) - (-2)) \times (-1) = (x - 1) \times (-1) = -(x - 1)$, která pro vstup $x = 2$ správně vrací -1 .

Průchod uzly je zajištěn cyklem `for (let i = 0; i < COLS; i++)`. Následně se načtou indexy z chromozomu do proměnných `in1 = chrom[i×3+0]`, `in2 = chrom[i×3+1]`, `fn = chrom[i×3+2]`. Načtené indexy jsou použity pro získání skutečných hodnot z pole `values[]`. Switch funkce pro parametr `fn` poté určí, jaká operace pro načtené hodnoty proběhne. Výsledek se uloží do `values[i+1]`.

Před aplikováním funkce nejprve proběhne kontrola vstupů, pokud `in1` nebo `in2` je $-\infty$ nebo NaN, tato hodnota se nahradí 0. Při operacích, které to vyžadují (dělení, odmocnina a logaritmus), jsou hodnoty dodatečně zkontrolovány, zda jsou pro danou funkci

validní, pokud ne, funkce vrací hodnotu 0. Po switch funkci je výsledek opět zkontrolován, zda nevrací $-\infty$ nebo NaN. Tyto kontroly byly implementovány za účelem zajištění validity výstupů, což zabraňuje chybám ve fitness funkci.

Index výstupu je vypočítán jako `idout = chrom[COLS × 3]` a výsledná hodnota je načtena z pole: `finalResult = values[idout]`. Funkce vrací hodnotu `finalResult`, která je znovu ošetřena proti $-\infty$ nebo NaN. Důkladné ošetření chyb funkce zaručuje stabilní běh evolučního algoritmu i při složitých evolucioních.

3.1.6 Mutace

Mutace, reprezentovaná funkcí `MutateChromozomu(chrom)`, je hlavním genetickým operátorem použitým v implementaci. Křížení (crossover) nebylo implementováno, jelikož se ve standardní implementaci CGP, podle Millera [22], nepoužívá. Křížení by mohlo narušit funkční bloky v grafové struktuře, a proto je samotná mutace často efektivnější.

Počet mutovaných genů závisí na hodnotě v reaktivní proměnné `PravdepodobnostMutace` nastavené uživatelem. Počet mutací se vypočítá podle vzorce:

$$\text{počet} = \max(1, \lfloor \text{délka_chromozomu} \times P_m \rfloor)$$

kde P_m je pravděpodobnost mutace (výchozí hodnota $P_m = 0,2$). Použití funkce $\max(1, \dots)$ zajišťuje, že vždy proběhne alespoň jedna mutace, i když vypočtená hodnota by byla nulová.

Podle typu genu se aplikují různé mutační operátory:

- **Mutate funkčního genu (fn)**
Původní funkce je náhodně zaměněna za jinou funkci z pole `vybraneFunkce`. Například funkce násobení nahradí sčítání.
- **Mutate vstupního genu (in1, in2)**
Je náhodně vybrán nový vstup z pole `PovolenaCisla`, do kterého jsou uloženy pouze validní hodnoty pro CGP – index vstupní proměnné, indexy konstant a indexy předchozích uzlů.
- **Mutate výstupního genu (output)**
Gen je zmutován na náhodné číslo z rozsahu $[0, \text{COLS})$.
- **Mutate evolvovatelné konstanty**
Mutací se přičte ke konstantě hodnota z Gaussova rozdělení $\mathcal{N}(0, \sigma = 0,5)$. Výsledná hodnota je omezena do rozsahu -100 až 100, aby nedošlo k numerické nestabilitě.

Funkce `MutateChromozomu(chrom)` zajišťuje explorační stavového prostoru se zachováním validity chromozomu.

3.1.7 Práce s konstantami

Aplikace podporuje dva režimy práce s konstantami, uživatel si v GUI volí jak jejich typ, tak i jejich množství. Oba režimy mají své výhody i nevýhody.

Fixní konstanty jsou generovány pomocí funkce `VygenerujFixniKonstanty`. Aplikace nabízí 4 konfigurace: 5, 10, 15 nebo 20 konstant. Pro množství 5, 10 a 15 jsou použity předdefinované hodnoty. V případě 20 konstant jsou hodnoty generovány s rovnoměrným rozložením v rozsahu od -10 do 10. Fixní konstanty jsou reprezentovány indexy 1001+ a jsou uloženy mimo chromozom.

Evolvovatelné konstanty jsou reprezentovány indexy 2000+ a jsou uloženy na konci chromozomu za výstupním uzlem. Jejich počet si uživatel volí z množiny $\{1, \dots, 5\}$. Inicializace evolvovatelných konstant probíhá pomocí Gaussova normálního rozložení, které je reprezentováno funkcí `gaussianRandom()`. Při inicializaci je použito rozdělení $\mathcal{N}(0, \sigma = 2)$, při mutaci pak $\mathcal{N}(0, \sigma = 0,5)$ pro jemnější změny. Gaussovo rozdělení bylo zvoleno díky jeho přirozenosti pro evoluční algoritmy – preferuje hodnoty blízko středu (0), ale zároveň nevyklučuje extrémní hodnoty.

Volba mezi režimy představuje kompromis. Fixní konstanty vedou k rychlejší a stabilnější konvergenci, ale za cenu omezené přesnosti – jsou vhodné pro jednoduché datasety s celočíselnými koeficienty. Evolvovatelné konstanty konvergují pomaleji, ale poskytují flexibilnější a přesnější aproximaci – umožňují aproximovat datasety s libovolnými reálnými koeficienty. Možnost volby mezi režimy umožňuje uživateli přizpůsobit algoritmus charakteru řešeného problému.

3.1.8 Generování funkčního předpisu

Po dokončení evoluce je potřeba výsledný chromozom převést na čitelný matematický zápis. Funkce `FunkcniPredpis(index)` slouží k převodu chromozomu do LaTeX notace pro zobrazení v GUI.

Funkce je rekurzivní, což znamená, že začíná u výstupního uzlu a postupně prochází chromozom zpět ke vstupům. Pro každý typ uzlu vrací odpovídající LaTeX řetězec. Základní případy rekurze jsou: index 0 vrací "**x**", indexy 1001+ vracejí hodnotu fixní konstanty a indexy 2000+ vracejí hodnotu evolvovatelné konstanty zaokrouhlenou na 3 desetinná místa.

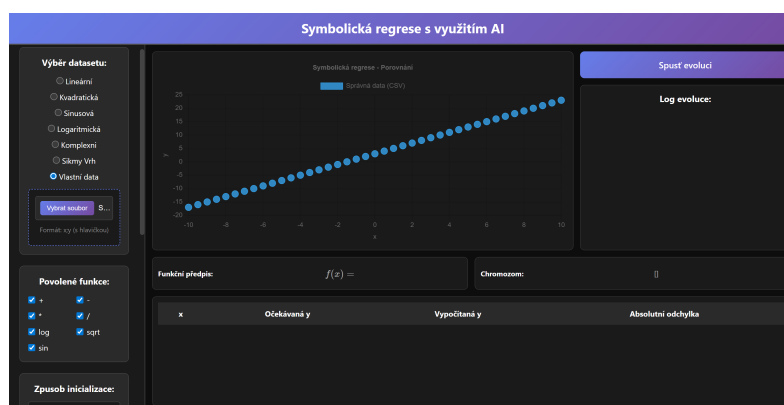
Pro každou operaci je LaTeX zápis jiný. Například pro sčítání je `in1 + in2`, pro dělení `\frac{in1}{in2}`, pro násobení `in1 \cdot in2`. U násobení a dělení je nutné kontrolovat, zda operandy obsahují `+` nebo `-`, a v případě potřeby vložit závorky. Tento mechanismus zajišťuje správné pořadí operací – například uzel s násobením a vstupy $(x + 1)$ a 2 bude vyhodnocen jako $(x + 1) \cdot 2$.

Výsledný LaTeX řetězec je na závěr předán knihovně MathJax 3, která ho vykreslí v profesionální typografické kvalitě přímo v prohlížeči. Rekursivní přístup zajišťuje korektní převod chromozomu i pro složité výrazy s libovolným počtem vnořených operací.

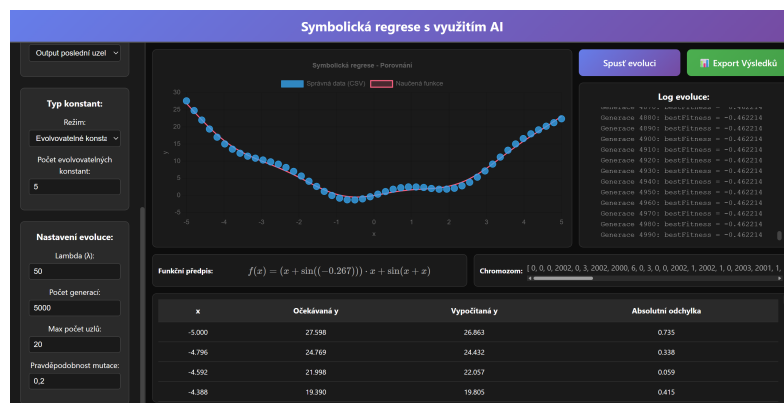
3.2 Grafické rozhraní

V následujících podkapitolách je popsána struktura grafického rozhraní, které je znázorněné na obrázcích 3.1 a 3.2, způsob vykreslování grafu a práce s daty.

3.2.1 Struktura rozhraní



Obrázek 3.1: Aplikace před spuštěním evoluce.



Obrázek 3.2: Aplikace po dokončení evoluce.

Levý panel obsahuje ovládací prvky pro nastavení parametrů. Uživatel může vybrat dataset (6 předpřipravených + možnost nahrát vlastní CSV soubor), povolit nebo zakázat matematické funkce, zvolit způsob inicializace, typ a množství konstant a parametry evoluce (λ , počet generací, COLS, pravděpodobnost mutace).

Pravá část je rozdělena do 3 řádků. První řádek obsahuje graf zobrazující načtená data z datasetu jako modré body a nalezenou funkci jako červenou křivku, ovládací tlačítka

pro spuštění a zastavení evoluce, tlačítko pro export výsledků a log fitness. Druhý řádek se skládá ze dvou bloků – funkčního předpisu a chromozomu. V bloku funkčního předpisu je zobrazen matematický vzorec naučené funkce, v bloku chromozomu jeho číselná reprezentace. Ve třetím řádku se nachází srovnávací tabulka s hodnotami x , očekávanou hodnotou y z datasetu, vypočítanou hodnotou $\hat{y}$ a absolutní odchylkou mezi nimi.

3.2.2 Vykreslování grafu

Graf je vykreslen funkcí `updateChart(showModel)`, která přijímá parametr určující, zda má být naučená funkce zobrazena. Pokud existuje předchozí instance grafu, je nejprve odstraněn zavoláním funkce `chartInstance.destroy()`, aby nedocházelo k duplicitám.

Funkce vytváří dva datasety pro Chart.js:

- **Dataset 1 – Scatter**

Tento dataset zobrazuje hodnoty z reaktivní proměnné `nactenaData`, která obsahuje dvojice $(x; y)$ z načteného datasetu a je vykreslen vždy. Data jsou reprezentována modrými body, přičemž každý bod odpovídá jednomu záznamu. Body nejsou spojeny křivkou.

- **Dataset 2 – Line**

Tento dataset je přidán pouze pokud `showModel === true`. Data jsou generována evaluací chromozomu (kapitola 3.1.5) pro každý bod z datasetu a následně seřazena podle hodnoty x pro plynulý průběh křivky. Výsledek je zobrazen jako červená křivka, která reprezentuje nalezenou funkci.

Pro vizualizaci byla podle návrhu zvolena knihovna Chart.js. Graf se automaticky aktualizuje při každé změně datasetu voláním `updateChart(false)`, což vykreslí pouze modré body. Během evoluce je funkce volána s parametrem `true` každých pět generací, čímž se přidá červená křivka znázorňující aktuální stav naučené funkce. Asynchronní volání pomocí `await Promise` zajišťuje, že rozhraní zůstává responzivní i během výpočetně náročné evoluce.

3.2.3 Načítání dat a reaktivita

Data jsou načítána dvěma způsoby. Předpřipravené datasety se načítají pomocí Fetch API z adresáře `/Datasets/`. Funkce `nactiDataset()` stahuje CSV soubor, normalizuje konce řádků, odstraňuje hlavičku a parsuje řádky rozdělením podle středníku. Vlastní data nahrává uživatel přes file input. Funkce `NahratiVlastniData()` validuje formát souboru, strukturu (dva sloupce oddělené středníkem s hlavičkou) a platnost číselných

hodnot. Data se ukládají do reaktivní proměnné `nactenaData`. V případě problému je uživatel informován popisem chyby.

Vue.js reaktivita je zajištěna pomocí `ref()` proměnných. Změna hodnoty například v `chromozom.value` automaticky vyvolá překreslení předpisu a grafu bez nutnosti manuálního volání render funkcí.

Datový tok je jednosměrný: uživatel $\rightarrow$ `ref` proměnné $\rightarrow$ evoluční algoritmus $\rightarrow$ `ref` výsledky $\rightarrow$ automatická aktualizace zobrazení. Evoluční algoritmus běží asynchronně pomocí `async/await` a `setTimeout(0)`. Každých pět generací je vyvolána aktualizace grafu s `await Promise`, což umožňuje prohlížeči zpracovat události a zachovat responzivitu rozhraní i během tisíců generací evoluce.

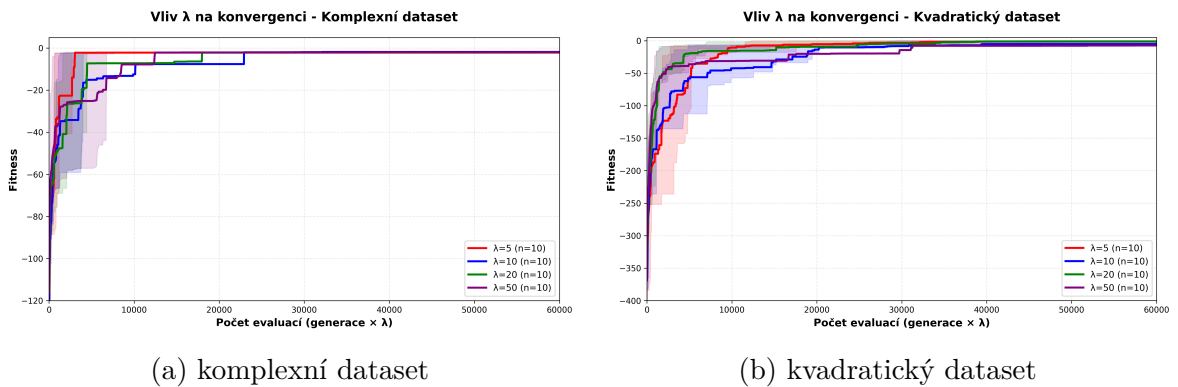
4 Vyhodnocení výsledků

4.1 Vliv parametrů evoluce na konvergenci

Pro následující experimenty (sekce 4.1.1–4.1.5) bylo použito výchozí nastavení: $\lambda = 20$, 10 uzlů, $P_m = 0,2$, náhodná inicializace a 15 fixních konstant. Každá konfigurace byla testována v 10 bězích na 5 datasetech. Grafy konvergence zobrazují vývoj průměrné fitness (plná čára = průměr, barevný pás = mezikvartilové rozpětí 25. – 75. percentil). Variabilitu doplňují boxploty finálních hodnot fitness. Reprezentativní grafy pro 2 datasety jsou u každého experimentu v textu, zbylé v příloze B.

4.1.1 Parametr lambda

Vliv parametru λ byl testován se 4 hodnotami (5; 10; 20; 50). Pro zajištění spravedlivého srovnání, kdy každá hodnota λ má k dispozici stejný výpočetní rozpočet, byl celkový počet evaluací fitness funkcí udržován konstantní – s rostoucím λ byl proporcionálně snižován počet generací tak, aby platilo $\lambda \times \text{generace} = 60\,000$ evaluací. Konkrétně: $\lambda = 5$ měla 12 000 generací, $\lambda = 10$ měla 6 000 generací, $\lambda = 20$ měla 3 000 generací a $\lambda = 50$ měla 1 200 generací. Na obrázku 4.1 jsou zobrazeny grafy konvergence pro komplexní a kvadratický dataset.



Obrázek 4.1: Grafy konvergence fitness v závislosti na počtu evaluací pro parametr λ

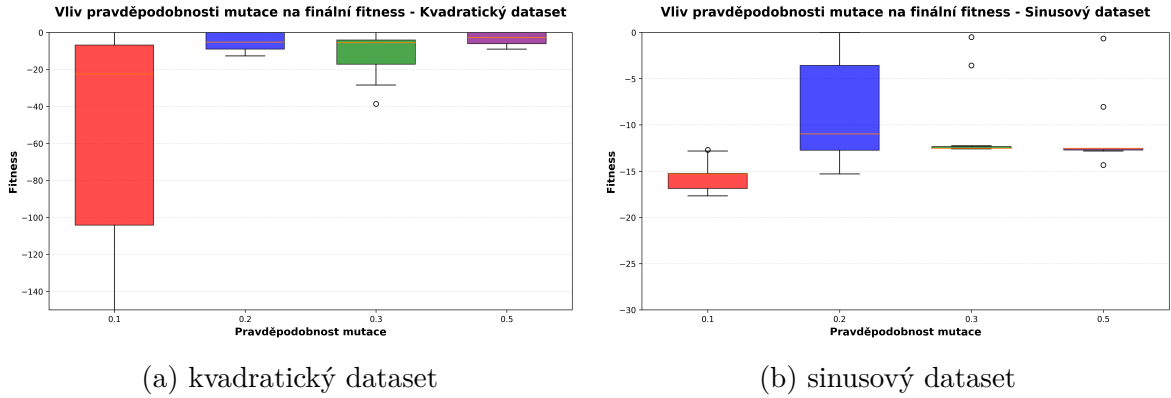
Grafy ukazují zajímavý vzor konvergence. U komplexního datasetu (obrázek 4.1a) nízká hodnota $\lambda = 5$ dosahuje nejlepších výsledků během prvních přibližně 12 000 evaluací.

Teprve poté ji ostatní hodnoty doženou a po přibližně 20 000 evaluacích všechny konvergují k prakticky stejnému výsledku. V případě kvadratického datasetu se jako nejlepší jeví hodnota $\lambda = 20$, která má společně s $\lambda = 50$ nejsilnější start, $\lambda = 50$ je však kolem 15 000 evaluací předechnána $\lambda = 5$, než se všechny víceméně vyrovnají.

Překvapivý výsledek u komplexního datasetu ($\lambda = 5$ vyhrála) lze vysvětlit tím, že více generací s nižší lambdou umožnilo jemnější ladění struktury, což bylo pro složitou funkci výhodnější. S ohledem na všechny datasety se ale jeví $\lambda = 20$ jako nejstabilnější volba.

4.1.2 Parametr pravděpodobnost mutace(P_m)

Vliv parametru pravděpodobnosti mutace byl testován se 4 hodnotami (0,1; 0,2; 0,3; 0,5). Na obrázku 4.2 jsou zobrazeny boxploty finálních fitness hodnot pro kvadratický a sinusový dataset.



Obrázek 4.2: Boxploty finálních hodnot fitness v závislosti na pravděpodobnosti mutace

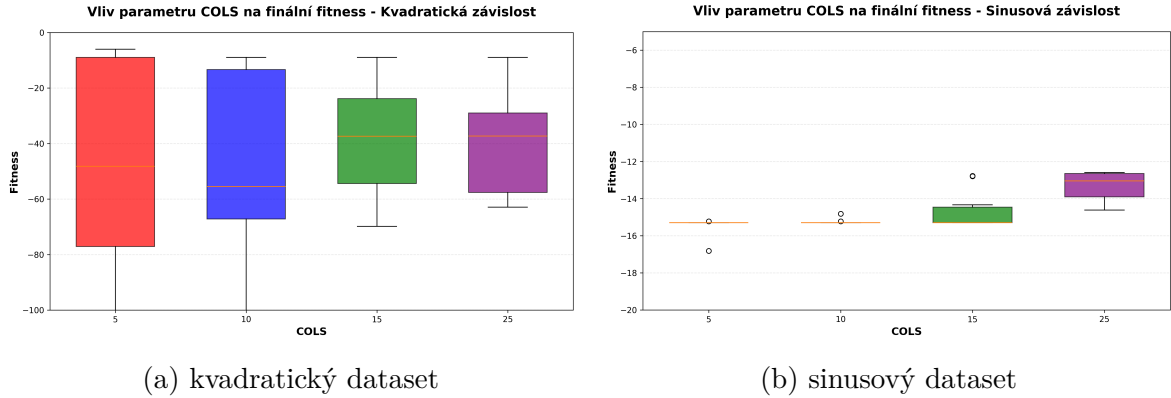
Na obrázku 4.2a je patrné, že nejvyšší hodnota $P_m = 0,5$ dosahuje nejlepší finální fitness (medián -3), zatímco nejnižší hodnota $P_m = 0,1$ vykazuje výrazně horší výsledky (medián -20) s velkou variabilitou. To naznačuje, že u polynomiálních funkcí je vyšší pravděpodobnost mutace výhodná pro explorační stavového prostoru a nalezení správné struktury výrazu. U sinusového datasetu (obrázek 4.2b) je trend opačný. Nejlepších výsledků dosahuje střední hodnota $P_m = 0.2$ (medián -10.5), zatímco nejnižší hodnota $P_m = 0,1$ (medián -16) i vyšší hodnoty $P_m = 0,3$ a $P_m = 0,5$ (medián -12 až -13) jsou horší. To lze vysvětlit tím, že periodické funkce vyžadují jemné doladění parametrů (amplitudy, frekvence, fázového posunu), což konzervativnější mutace lépe umožňuje. Příliš nízká mutace ($P_m = 0,1$) naopak nedovoluje dostatečné prozkoumání stavového prostoru.

Experiment ukázal, že optimální hodnota pravděpodobnosti mutace závisí na typu řešeného problému. Pro složitější struktury je výhodnější vyšší hodnota díky větší exploraci,

zatímco pro problémy vyžadující jemné doladění parametrů je vhodnější střední až nižší hodnota. Z praktického hlediska se jako univerzální volba osvědčila hodnota $P_m = 0,2$.

4.1.3 Parametr COLS (maximální počet uzlů)

Vliv parametru maximálního počtu uzlů byl testován se čtyřmi hodnotami (5; 10; 15; 25). Na obrázku 4.3 jsou zobrazeny boxploty finálních fitness hodnot pro kvadratický a sinusový dataset.



Obrázek 4.3: Boxploty finální fitness v závislosti na parametru COLS

Na obrázku 4.3a je patrné, že vyšší hodnoty $COLS = 15$ a $COLS = 25$ dosahují lepších mediánů finální fitness (přibližně -35 až -38) než nižší hodnoty $COLS = 5$ a $COLS = 10$ (medián kolem -50 až -55). Nízká hodnota $COLS = 5$ poskytuje algoritmu nedostatečný prostor pro vytvoření složitějšího výrazu, což vede k horším výsledkům. U sinusového datasetu (obrázek 4.3b) je trend podobný. Vyšší hodnota $COLS = 25$ dosahuje nejlepší finální fitness (medián -13) oproti nižším hodnotám $COLS=5, 10$ a 15 s mediány kolem -15 . To potvrzuje, že u složitějších funkcí (polynomiálních i periodických) je výhodné poskytnout algoritmu větší prostor pro vytváření expresivnějších struktur.

Zajímavým zjištěním je, že přestože vyšší $COLS$ hodnoty vedou k lepším výsledkům, algoritmus ve skutečnosti využívá pouze malou část dostupných uzlů. V následující tabulce 4.1 je znázorněno využití dostupných uzlů pro kvadratický dataset se všemi testovanými hodnotami $COLS$. Minimální počet uzlů potřebný pro správné vyjádření kvadratické funkce $(x - 2)^2 + 3$ je přibližně 3.

COLS	5	10	15	25
Aktivní uzly	2,2	2,5	2,9	2,8

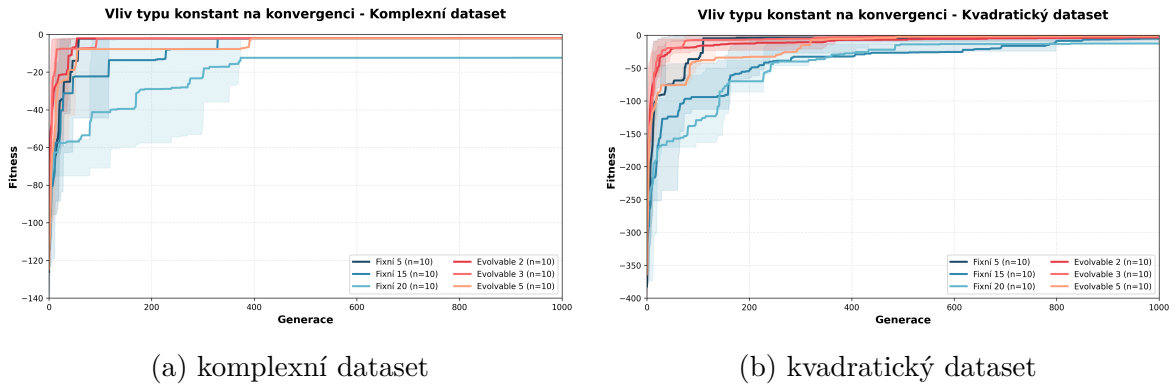
Tabulka 4.1: Využití uzlů chromozomu pro kvadratický dataset (průměr z 10 běhů)

Z tabulky je patrné, že algoritmus využíval prakticky totožné množství uzlů při všech omezeních. Toto ukazuje schopnost algoritmu předcházet bloatu. Penalizace složitosti

zajišťuje, že algoritmus preferuje jednodušší funkce, které dokáží data popsat s dostatečnou přesností. Z praktického hlediska se jako optimální jeví hodnota $COLS = 15$, která poskytuje dostatečný prostor pro expresivitu při zachování efektivity výpočtu.

4.1.4 Porovnání vlivu nastavení konstant

Vliv typu konstant byl testován s fixními konstantami (5; 15; 20) a evolvovatelnými konstantami (2; 3; 5). Různými odstíny červené jsou v grafech reprezentovány právě evolvovatelné konstanty a fixní konstanty jsou reprezentovány odstíny modré. Toto umožní porovnat jak typ konstant obecně, tak právě i jejich množství. Na obrázku 4.4 jsou zobrazeny grafy konvergence pro komplexní a kvadratický dataset.



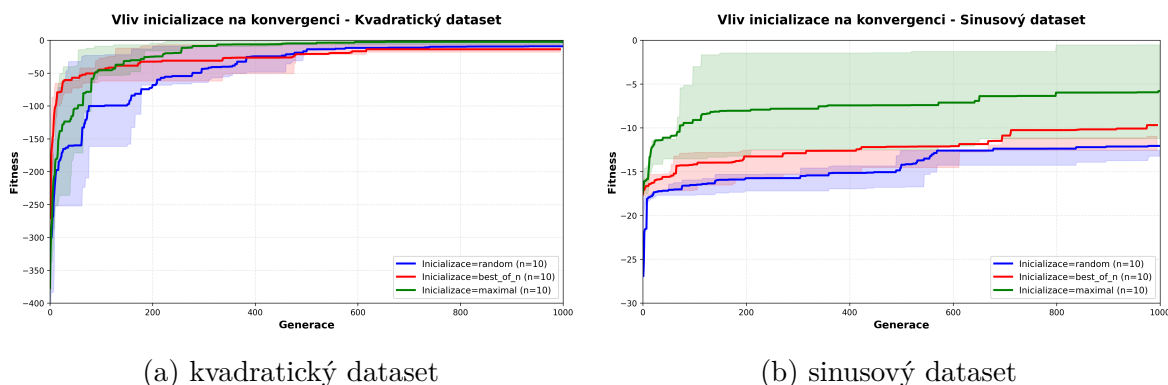
Obrázek 4.4: Grafy konvergence fitness demonstrující vliv nastavení konstant

Na obou grafech lze vidět, že evolvovatelné konstanty jsou výrazně lepší, jelikož umožňují dosáhnout hodnot, které přesně algoritmus vyžaduje. Fixní konstanty vybírají pouze z předem vybrané množiny, proto nejsou schopné tak přesně vyjádřit a efektivně měnit funkční předpis. Překvapivým zjištěním je, že v rámci fixních konstant dosahuje nejlepších výsledků nejmenší testovaná množina (5 konstant). Tento jev lze vysvětlit velikostí stavového prostoru – s menším počtem konstant je prostor možných řešení menší, což umožňuje algoritmu efektivněji konvergovat ke správnému řešení. Zároveň se ukázalo, že pět dobře zvolených konstant postačuje pro sestavení složitějších výrazů.

Z praktického hlediska se evolvovatelné konstanty jeví jako nejlepší volba pro všechny problémy. Přestože vyžadují více času a mají vyšší výpočetní náročnost, rozdíl v kvalitě řešení oproti fixním konstantám je výrazný. Jako optimální se ukázal počet 2–3 konstant, představující kompromis mezi expresivitou modelu a výpočetní náročností.

4.1.5 Porovnání strategií inicializace

Vliv způsobu inicializace na konvergenci byl testován se třemi způsoby inicializace výchozího chromozomu: náhodná, s maximálním propojením a *Best of N*. Na obrázku 4.5 jsou zobrazeny grafy konvergence pro kvadratický a sinusový dataset.



Obrázek 4.5: Grafy konvergence fitness demonstrující vliv typu inicializace

U všech testovaných datasetů dosahuje nejlepších finálních výsledků inicializace s maximálním propojením, přestože na začátku je u některých datasetů výrazně horší než metoda *Best of N*. Toto chování lze vysvětlit tím, že maximální inicializace garantuje aktivaci všech uzlů chromozomu již od začátku evoluce. Ačkoliv maximálně propojené uzly vedou k horší počáteční fitness, strukturovaná topologie umožňuje algoritmu efektivněji explarovat stavový prostor, což přináší nejlepší výsledky. Metoda *Best of N* sice zaručí dobré řešení již na začátku, ale většinou se jedná o jednoduchý výraz, který se následně obtížně upravuje do přesnější formy.

Experiment prokázal jasný vliv typu inicializace na kvalitu konvergence algoritmu. Jako nejefektivnější metoda se osvědčila inicializace s maximálním propojením, následovaná přístupem *Best of N*. U varianty s maximálním propojením však existuje riziko již zmíněného bloatu, neboť proces začíná se všemi uzly v aktivním stavu. Volba metody tedy závisí na prioritách: pokud je cílem nalezení nejpřesnějšího řešení i přes riziko bloatu, je optimální volbou maximální propojení. V případě, že je prioritou prevence bloatu na úkor mírně nižší kvality výsledku, je vhodnější zvolit inicializaci *Best of N*.

4.1.6 Rekonstrukce testovacích datasetů

Kromě analýzy vlivu jednotlivých parametrů bylo testováno, zda algoritmus dokáže všechny datasety přesně zrekonstruovat (dosáhnout kvalitní fitness) při použití optimální konfigurace parametrů, která byla stanovena na základě předchozích experimentů. Této shody bylo úspěšně dosaženo u všech pěti testovaných datasetů. Screenshoty grafického rozhraní s výslednými funkcemi a vizualizací shody společně se souhrnnou tabulkou výsledných hodnot MSE jsou pro každý dataset uvedeny v příloze C.

4.2 Reálná aplikace – šikmý vrh

Pro ověření schopnosti algoritmu pracovat s reálnými fyzikálními daty byla aplikace testována na problému šikmého vrhu. Dataset byl vygenerován se šumem (simulace

reálného měření) a následujícími parametry: počáteční rychlost $v_0 = 20$ m/s, úhel vrhu $\theta = 45^\circ$ a tíhové zrychlení $g = 9,8$ m/s². Teoretická trajektorie šikmého vrhu je popsána rovnicí:

$$y(x) = x \cdot \tan(\theta) - \frac{g \cdot x^2}{2 \cdot v_0^2 \cdot \cos^2(\theta)}$$

Po dosazení parametrů získáme cílovou funkci, kterou měl algoritmus objevit:

$$y(x) = x - 0,0245x^2$$

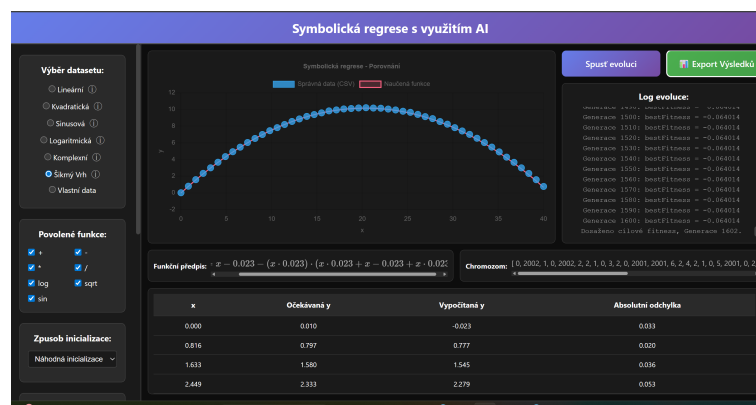
Algoritmus byl spuštěn s výchozím nastavením. Výraz, kterým algoritmus dokázal zrekonstruovat data, měl po algebraickém zjednodušení následující tvar:

$$f(x) = 1,00053x - 0,0241x^2 - 0,023$$

Koeficient	Teoreticky	Nalezeno	Relativní chyba
Lineární člen (x)	1,0	1,00053	0,05 %
Kvadratický člen (x ²)	−0,0245	−0,0241	2,0 %
Konstantní člen	0	−0,023	—

Tabulka 4.2: Porovnání koeficientů nalezené a teoretické funkce

Porovnání v tabulce 4.2 ukazuje vynikající shodu s teoretickým modelem. Lineární koeficient byl nalezen s přesností 99,95 %, kvadratický koeficient s přesností 98 %. Konstantní člen −0,023 je artefakt evoluce, kdy algoritmus našel, že malý offset může lokálně snížit fitness na některých bodech datasetu. Finální MSE dosáhla hodnoty 0,0048, což odpovídá průměrné absolutní odchylce přibližně 0,07 m na bodech trajektorie.



Obrázek 4.6: Screenshot aplikace při zrekonstruování šikmého vrhu

Graf na obrázku 4.6 zobrazuje téměř dokonalou shodu nalezené funkce (červená křivka) s naměřenými daty (modré body). Tento experiment potvrzuje schopnost mého algoritmu rekonstruovat fyzikální zákony z dat bez předem dané struktury funkce.

5 Závěr a diskuze

Cílem této práce bylo vytvořit interaktivní webovou aplikaci pro symbolickou regresi využívající kartézské genetické programování. V této kapitole jsou shrnuty dosažené výsledky, diskutována omezení implementovaného řešení a navrženy možnosti budoucího rozšíření.

5.1 Shrnutí dosažených cílů

Všechny cíle stanovené v kapitole 2.1 byly úspěšně splněny. Byl implementován evoluční algoritmus založený na CGP využívající strategii $(1+\lambda)$, kde chromozom je reprezentován jako jednorozměrné pole celých čísel. Fitness funkce založená na MSE obsahuje tři penalizační mechanismy předcházející bloatu a overfittingu.

Aplikace nabízí intuitivní GUI umožňující sledovat průběh evoluce v reálném čase. Byly implementovány tři strategie inicializace (náhodná, s maximálním propojením, výběr nejlepšího z N) a dva režimy konstant (fixní a evolvovatelné). Systematické experimenty na pěti datasetech a reálném problému šikmého vrhu potvrdily funkčnost algoritmu.

Nad rámec stanovených cílů byly implementovány funkce pro export výsledků do CSV souboru a především možnost nahrání vlastních dat uživatelem. Tato rozšíření významně zvyšují praktickou využitelnost aplikace, jelikož uživatel může analyzovat vlastní experimentální data a hledat v nich dosud neznámé matematické vztahy.

5.2 Zhodnocení výsledků

Experimentální vyhodnocení přineslo několik důležitých zjištění. Evolvovatelné konstanty výrazně překonávají fixní ve všech scénářích. Inicializace s maximálním propojením dosahuje nejlepších finálních výsledků. Penalizace složitosti účinně brání nadměrnému růstu výrazů – algoritmus konzistentně preferoval jednodušší řešení.

Výsledky experimentů jsou v souladu s teoretickými poznatky o CGP prezentovanými Millerem [22]. Potvrdilo se, že mutace jako jediný genetický operátor postačuje k efektivní exploraci stavového prostoru, což odpovídá standardnímu přístupu v CGP. Experimenty s parametrem λ ukázaly, že střední hodnoty (kolem 20) poskytují nejstabilnější

konvergenci, což koresponduje s doporučeními v literatuře pro CGP. Rovněž se potvrdil fenomén neutrality – mnoho mutací nemění fenotyp chromozomu, což umožňuje algoritmu procházet stavovým prostorem bez okamžité ztráty fitness.

Algoritmus úspěšně zrekonstruoval všech pět testovacích datasetů s MSE pod 0,015. Nejvýznamnějším výsledkem je rekonstrukce trajektorie šikmého vrhu, kde algoritmus objevil funkci $f(x) = 1,00053x - 0,024x^2 - 0,023$ odpovídající teoretickému modelu s relativní chybou pod 2 %. Tento výsledek demonstruje sílu programu a jeho praktickou využitelnost pro objevování matematických vztahů z nahraných experimentálních dat.

5.3 Omezení práce

Implementované řešení má několik omezení. Nejvýznamnějším z nich je podpora pouze jednorozměrné regrese – aplikace pracuje s jednou vstupní proměnnou x . Sada funkcí je omezená na základní operace a nejčastěji používané funkce (sin, log, sqrt) postačující pro běžné závislosti, chybí exponenciála, kosinus nebo obecné mocniny.

Aplikace běží v prohlížeči pomocí JavaScriptu, což omezuje výpočetní výkon oproti nativním implementacím. Chybí paralelizace, která by běh algoritmu zrychlila – nyní výpočty probíhají v jednom vlákne. V algoritmu také chybí automatická simplifikace výsledných výrazů, která by je zkrátila a udělala jednodušší.

5.4 Možnosti budoucího rozšíření

Prioritním rozšířením by byla podpora více vstupních proměnných, což by umožnilo řešit širší spektrum úloh včetně komplexnějších vícerozměrných problémů. Paralelizace pomocí Web Workers by urychlila výpočty, jelikož evaluace fitness potomků by byla nezávislá operace. Automatická simplifikace výrazů pomocí knihovny pro symbolickou matematiku by zlepšila čitelnost nalezených funkcí.

Rozšíření sady funkcí o exponenciálu, kosinus a obecné mocniny by zvýšilo expresivitu algoritmu. Zajímavým směrem by bylo srovnání s knihovnou gplearn na identických datasetech, které by umožnilo objektivní porovnání výkonu. Dále by se dala porovnat rychlost konvergence algoritmu vůči jiným metodám prohledávání stavového prostoru, jako je náhodné prohledávání či horolezecký algoritmus.

Závěrem lze konstatovat, že vytvořená aplikace splňuje všechny stanovené cíle a představuje funkční nástroj pro interaktivní symbolickou regresi. Díky webovému rozhraní je snadno dostupná bez nutnosti instalace a umožňuje experimentovat s parametry v reálném čase. Práce má kromě praktického využití i vzdělávací potenciál, neboť vizualizace evolučního procesu usnadňuje pochopení principů kartézského genetického programování.

Literatura

- [1] GeeksforGeeks. Introduction to hill climbing | artificial intelligence, 2025. Online. Dostupné z: <https://www.geeksforgeeks.org/artificial-intelligence/introduction-hill-climbing-artificial-intelligence/> [cit. 2025-10-26].
- [2] KHAN Najeeb. A parallel implementation of the covariance matrix adaptation evolution strategy, 2015. Online. Dostupné z: <https://doi.org/10.48550/arXiv.1805.11201> [cit. 2025-10-27].
- [3] BALACHANDRAN Prasanna V., XUE Dezhen, THEILER James, and LOOKMAN Turab. Schematic showing the trade-off between exploration and exploitation: A hypothetical fitness landscape, 2016. Online obrázek. Dostupné z: https://www.researchgate.net/figure/Schematic-showing-the-trade-off-between-exploration-and-exploitation-A-hypothetical_fig2_291419889 [cit. 2025-12-08].
- [4] SASTRY Kumara, GOLDBERG David E., and KENDALL Graham. Illustration of examples of one point, two points, and uniform crossover methods. In Edmund K. BURKE and Graham KENDALL, editors, *Search Methodologies: Introductory Tutorials in Optimization and Decision Support Techniques*, pages 98–125. Springer, Boston, MA, 2005. Online obrázek. Dostupné z: <https://www.graham-kendall.com/papers/sgk2005.pdf> [cit. 2025-12-09].
- [5] SHAHIN Mohamed A. Typical example of genetic programming (gp) tree representation for the function $[(4x^1 - 1)/(x^2 - 3x + 3)]^2$. 130, 11 2014. doi: 10.1016/j.gsf.2014.10.002. Online obrázek. In: State-of-the-art review of some artificial intelligence applications in pile foundations. Dostupné z: https://www.researchgate.net/figure/Typical-example-of-genetic-programming-GP-tree-representation-for-the-function-4Dx-1-x_fig3_267626837 [cit. 2025-01-20].
- [6] MIRAGAIA Rolando, FERNÁNDEZ DE VEGA Francisco, and REIS Gustavo. A cartesian genetic programming (cgp) generic example of a genotype and its corresponding phenotype, 3 2021. Online obrázek. In: Evolving a Multi-Classifer System for Multi-Pitch Estimation of Piano Music and Beyond: An Application of Cartesian Genetic Programming. Dostupné z: <https://www.researchgate.n>

et/figure/A-Cartesian-genetic-programming-CGP-generic-example-of-a-genotype-and-its-corresponding_fig4_350362418 [cit. 2025-12-14].

- [7] KALVODA Tomáš. Bi-jul – Úvod do umělé inteligence: 11. přednáška – optimalizace, 2025. Online. Dostupné z: <https://courses.fit.cvut.cz/BI-JUL/tutorials/bi-jul-11.html> [cit. 2025-10-10].
- [8] JOHNSON Steven G. A brief overview of optimization problems, 2008. Online. Dostupné z: <https://math.mit.edu/~stevenj/18.335/fall08/optimization.pdf> [cit. 2025-10-10].
- [9] DONG Junlan and ZHONG Jinghui. Recent advances in symbolic regression. 57: 1–37, 2025. Online. Dostupné z: <https://dl.acm.org/doi/10.1145/3735634> [cit. 2025-10-12].
- [10] TIAN Yuan et al. Interactive symbolic regression with co-design mechanism through offline reinforcement learning. 16, 2025. Online. Dostupné z: <https://doi.org/10.1038/s41467-025-59288-y> [cit. 2025-10-12].
- [11] VIRGOLIN Marco and PISSIS Solon P. Symbolic regression is np-hard. 2022. Online. Dostupné z: <https://arxiv.org/abs/2207.01018> [cit. 2025-10-12].
- [12] Code Labs Academy. Gradientní sestup a stochastický gradientní sestup ve strojovém učení, 2024. Online. Dostupné z: <https://codelabsacademy.com/cs/blog/gradient-descent-and-stochastic-gradient-descent-in-machine-learning> [cit. 2025-12-15].
- [13] ZABINSKY Zelda B. Random search algorithms, 2009. Online. Dostupné z: <https://courses.washington.edu/inde510/516/AdapRandomSearch4.05.2009.pdf> [cit. 2025-10-26].
- [14] SOLIS Francisco J. and WETS Roger J-B. Minimization by random search techniques. 6(1):19–30, 1981. Online. Dostupné z: https://www.math.ucdavis.edu/~rjbw/mypage/Miscellaneous_files/randSearch.pdf [cit. 2025-10-26].
- [15] SEKANINA Lukáš et al. *Evoluční hardware: Od automatického generování patentovatelných invencí k sebemodifikujícím se strojům*. Academia, Praha, 2009. ISBN 978-80-200-1729-1.
- [16] EIBEN A. E. and SMITH James E. *Introduction to Evolutionary Computing*. Springer, 2nd edition, 2015. ISBN 978-3-662-44874-8. Online. Dostupné z: https://uomustansiriyah.edu.iq/media/lectures/6/6_2024_10_09!10_32_54_AM.pdf [cit. 2025-12-08].
- [17] BÄCK Thomas and SCHWEFEL Hans-Paul. An overview of evolutionary algorithms for parameter optimization. 1(1):1–23, 1993. doi: 10.1162/evco.1993.1.1.1.

Online. Dostupné z: <https://web.mit.edu/6.034/www/6.s966/baeck-ec93.pdf> [cit. 2025-12-08].

- [18] FOGEL Gary B., FOGEL David, and FOGEL Lawrence. Evolutionary programming. 6(4):1818, 2011. doi: 10.4249/scholarpedia.1818. Online. Dostupné z: http://www.scholarpedia.org/article/Evolutionary_programming [cit. 2025-12-07]. Revision #137293.
- [19] HOLLAND John H. *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence*. MIT Press, Cambridge, MA, 1992. First edition 1975, University of Michigan Press. Online. Dostupné z: http://repo.darmajaya.ac.id/3794/1/Adaptation%20in%20Natural%20and%20Artificial%20Systems_%20An%20Introductory%20Analysis%20with%20Applications%20to%20Biology%2C%20Control%2C%20and%20Artificial%20Intelligence%20%28A%20Bradford%20Book%29%20%28%20PDFDrive%20%29.pdf [cit. 2025-12-08].
- [20] KOZA John R. *Genetic Programming: On the Programming of Computers by Means of Natural Selection*. MIT Press, Cambridge, MA, 1992. ISBN 0-262-11170-5.
- [21] POLI Riccardo, LANGDON William B., and MCPHEE Nicholas Freitag. *A Field Guide to Genetic Programming*. Lulu Enterprises, Morrisville, NC, USA, 2008. ISBN 978-1-4092-0073-4. Online. Dostupné z: http://www0.cs.ucl.ac.uk/staff/W.Langdon/ftp/papers/poli08_fieldguide.pdf [cit. 2025-12-14].
- [22] MILLER Julian F. Cartesian genetic programming. In Julian F. MILLER, editor, *Cartesian Genetic Programming*, pages 17–34. Springer-Verlag, Berlin; Heidelberg, 2011. doi: 10.1007/978-3-642-17310-3_2. Online. Dostupné z: https://link.springer.com/chapter/10.1007/978-3-642-17310-3_2 [cit. 2025-12-14].
- [23] MAKKE Nadim and CHAWLA Sanjay. Interpretable scientific discovery with symbolic regression: a review. 57, 2024. doi: 10.1007/s10462-023-10622-0. Online. Dostupné z: <https://link.springer.com/article/10.1007/s10462-023-10622-0> [cit. 2025-12-08].
- [24] KURUI Moses. Mastering regression metrics: R^2 , mse, mae explained, 2025. Online. Dostupné z: <https://codepointtech.com/mastering-regression-metrics-r%C2%B2-mse-mae-explained/> [cit. 2025-01-13].
- [25] BROWNLEE Jason. Regression metrics for machine learning, 2021. Online. Dostupné z: <https://machinelearningmastery.com/regression-metrics-for-machine-learning/> [cit. 2025-01-13].

- [26] ALDEIA Guilherme Seidyo Imai. Current challenges of symbolic regression: Optimization, selection, model simplification, and benchmarking, 2025. Online. Dostupné z: <https://arxiv.org/abs/2512.01682v1> [cit. 2025-12-08].
- [27] ZUO Xiangwu and JIANG Anxiao. Symbolic modeling for financial asset pricing. 11:100150, 2025. doi: 10.1016/j.jfds.2025.100150. Online. Dostupné z: <https://doi.org/10.1016/j.jfds.2025.100150> [cit. 2025-12-13].

Přílohy

A Struktura souboru

Aplikace je organizována do následující adresářové struktury:

```
SOC-REGRESE/  
  public/  
    Datasets/  
      KomplexniDataset.csv  
      KvadratickaZavislost.csv  
      LinearniZavislost.csv  
      LogaritmickaZavislost.csv  
      SikmyVrh.csv  
      SinusovaZavislost.csv  
  src/  
    assets/  
      scripts/  
      vue.svg  
    components/  
      App.vue  
    main.js  
    style.css  
  scripty_na_grafy/  
    analyza_porovnani_cols.py  
    analyza_porovnani_inicializac.py  
    analyza_porovnani_konstant.py  
    analyza_porovnani_lambd.py  
    analyza_porovnani_mutace.py  
    boxplot_cols.py  
    boxplot_mutace.py  
  automatickeBehy.py  
  index.html  
  package.json  
  package-lock.json  
  README.md  
  vite.config.js
```

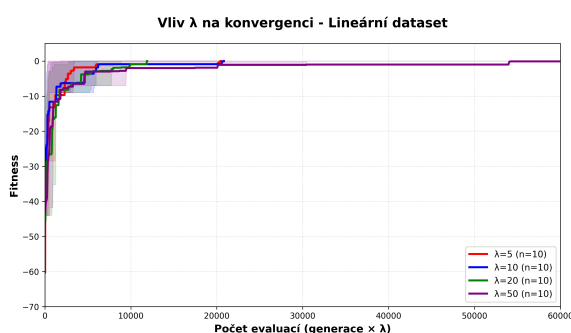
Popis hlavních komponent:

- `src/components/App.vue` – hlavní Vue.js komponenta obsahující implementaci evolučního algoritmu a grafického rozhraní
- `src/main.js` – vstupní bod aplikace, inicializace Vue
- `public/Datasets/` – adresář s předpřipravenými datasety pro testování
- `scripty_na_grafy/` – Python skripty pro generování grafů a vyhodnocení experimentů
- `automatickeBehy.py` – skript pro automatizované spouštění experimentů
- `vite.config.js` – konfigurace build nástroje Vite
- `package.json` – definice závislostí projektu (Vue.js, Chart.js)
- `README.md` – kompletní dokumentace projektu včetně návodu na instalaci, použití a přehledu výsledků experimentů

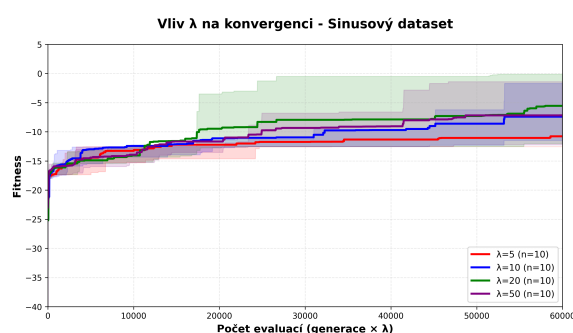
B Kompletní výsledky

Následující grafy doplňují měření uvedená v hlavní části práce pro zbývající datasety.

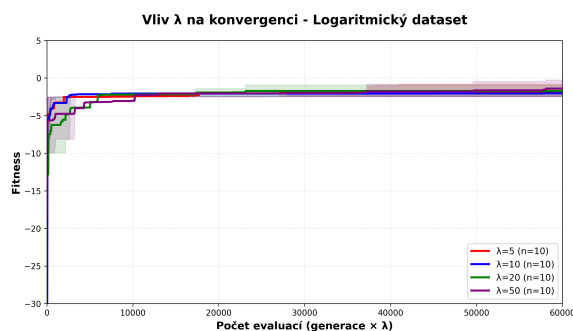
Parametr lambda



(a) lineární dataset



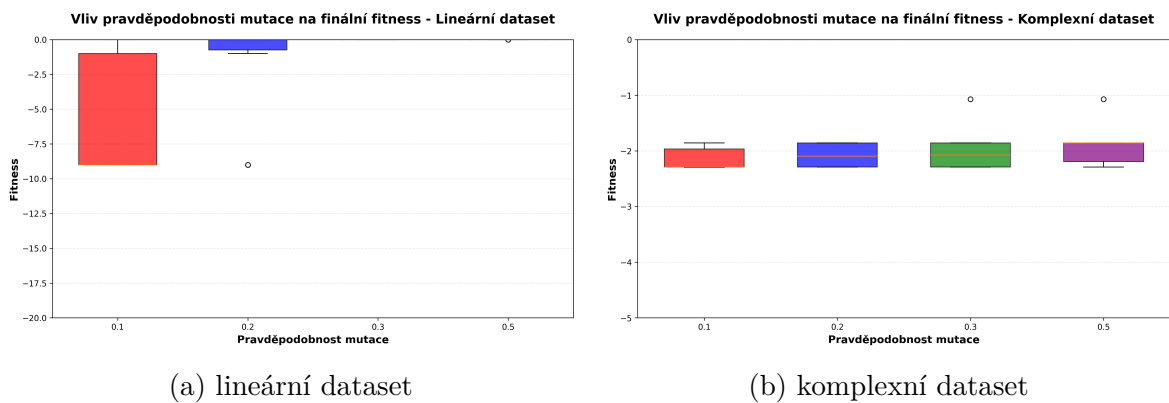
(b) sinusový dataset



(c) logaritmický dataset

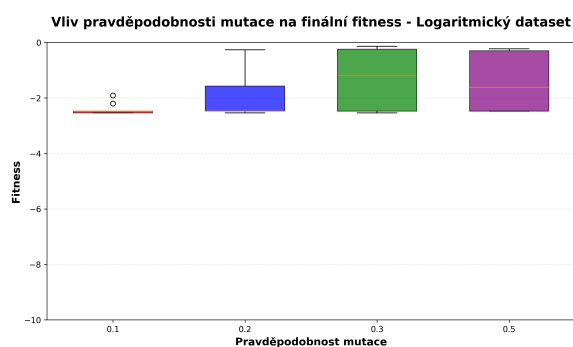
Obrázek B.1: Doplňující grafy konvergence fitness v závislosti na počtu evaluací pro parametr λ

Parametr pravděpodobnost mutace (P_m)



(a) lineární dataset

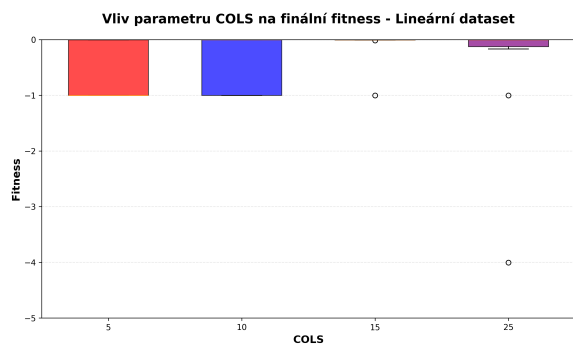
(b) komplexní dataset



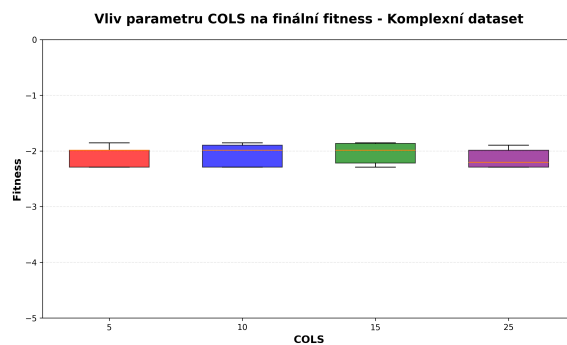
(c) logaritmický dataset

Obrázek B.2: Doplňující grafy boxploty s finální fitness pro parametr P_m (pravděpodobnost mutace)

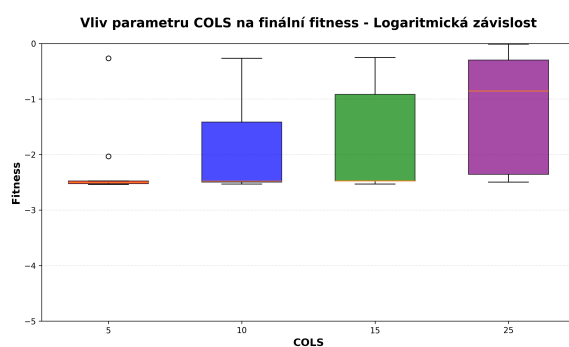
Parametr COLS (maximální počet uzlů)



(a) lineární dataset



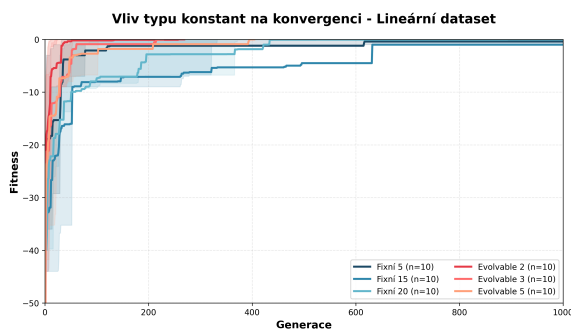
(b) komplexní dataset



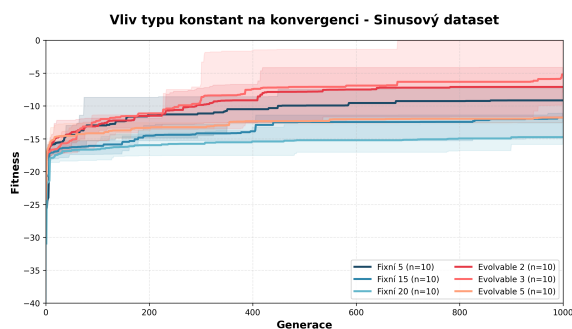
(c) logaritmický dataset

Obrázek B.3: Doplnující grafy boxploty s finální fitness pro parametr COLS

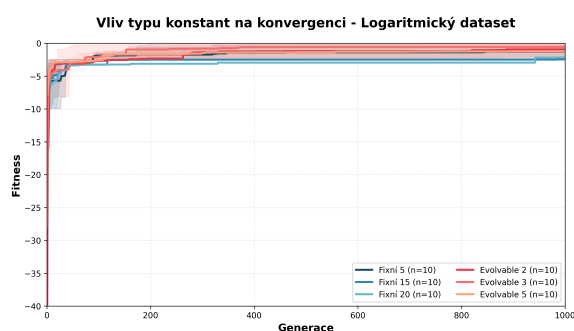
Porovnání vlivu nastavení konstant



(a) lineární dataset



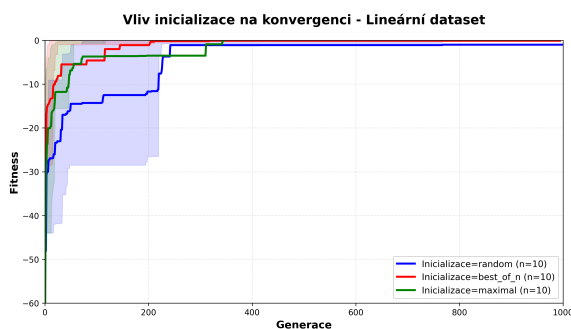
(b) sinusový dataset



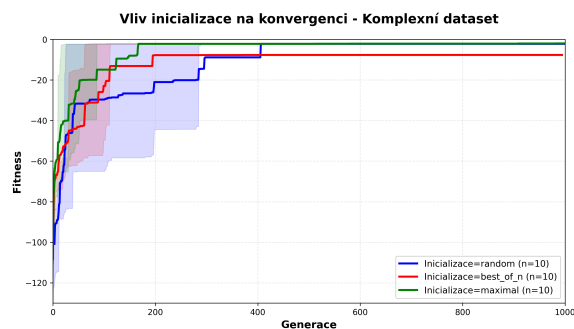
(c) logaritmický dataset

Obrázek B.4: Doplňující grafy konvergence fitness v závislosti na nastavení konstant

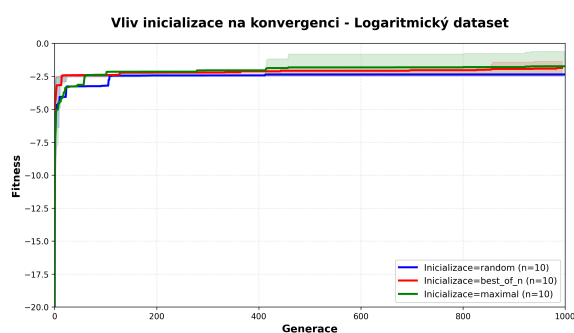
Porovnání strategií inicializace



(a) lineární dataset



(b) komplexní dataset



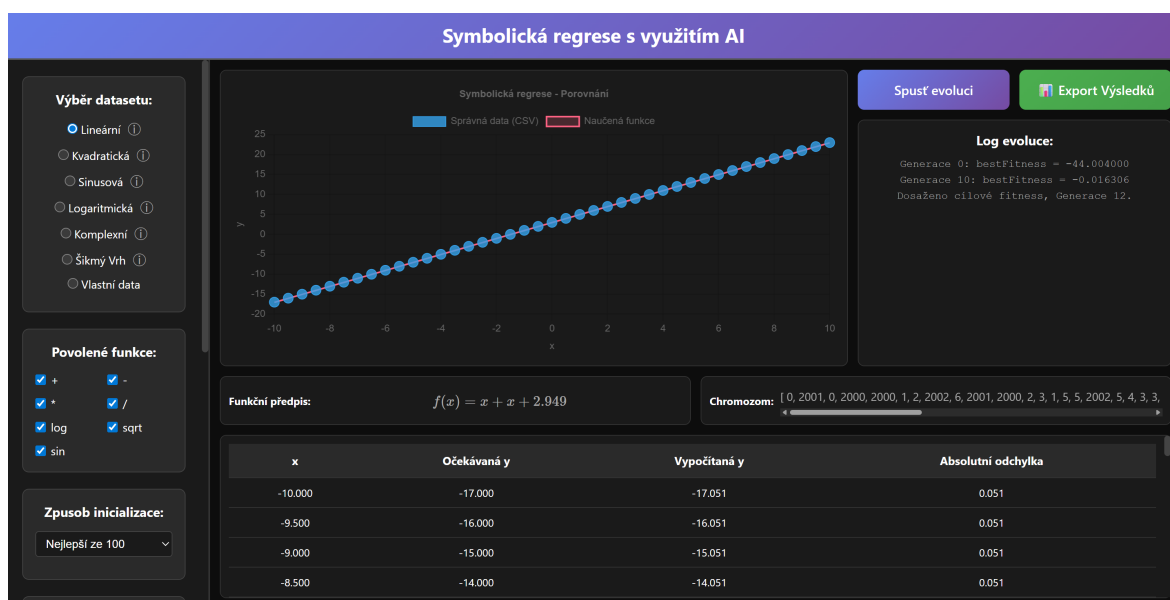
(c) logaritmický dataset

Obrázek B.5: Doplnující grafy konvergence fitness v závislosti na zvoleném typu inicializace

C Rekonstrukce testovacích datasetů

V této příloze jsou dokumentovány výsledky rekonstrukce 5 základních datasetů při použití optimálního nastavení parametrů ($\lambda = 20$, $COLS = 10$, $P_m = 0,2$, inicializace *Best of N*, 3 evolvovatelné konstanty). Maximální počet generací byl nastaven na 10 000.

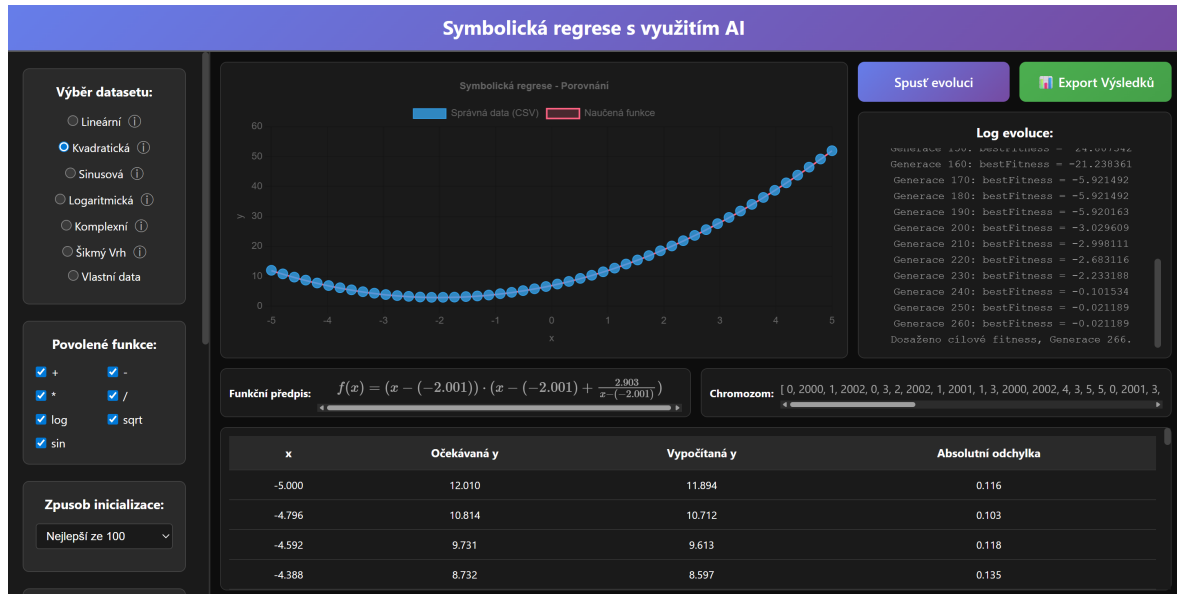
Lineární dataset



Obrázek C.1: Rekonstrukce lineární funkce.

- Cílový vzorec: $y = 2x + 3$
- Nalezený vzorec: $f(x) = x + x + 2,949$
- Po algebraické simplifikaci: $f(x) = 2x + 2,949$
- Finální hodnota MSE: 0,0026

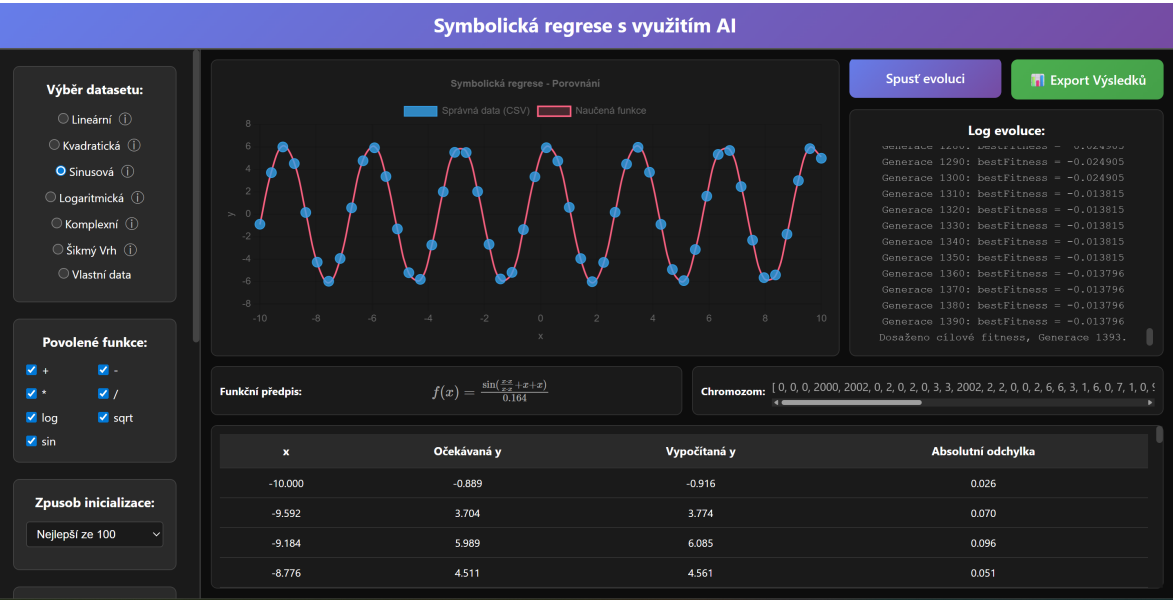
Kvadratický dataset



Obrázek C.2: Rekonstrukce kvadratické funkce.

- Cílový vzorec: $y = (x + 2)^2 + 3$
- Nalezený vzorec: $f(x) = (x - (-2,001)) \cdot (x - (-2,001) + \frac{2,903}{x - (-2,001)})$
- Po algebraické simplifikaci: $f(x) = (x + 2,001)^2 + 2,903$
- Finální hodnota MSE: 0,0081

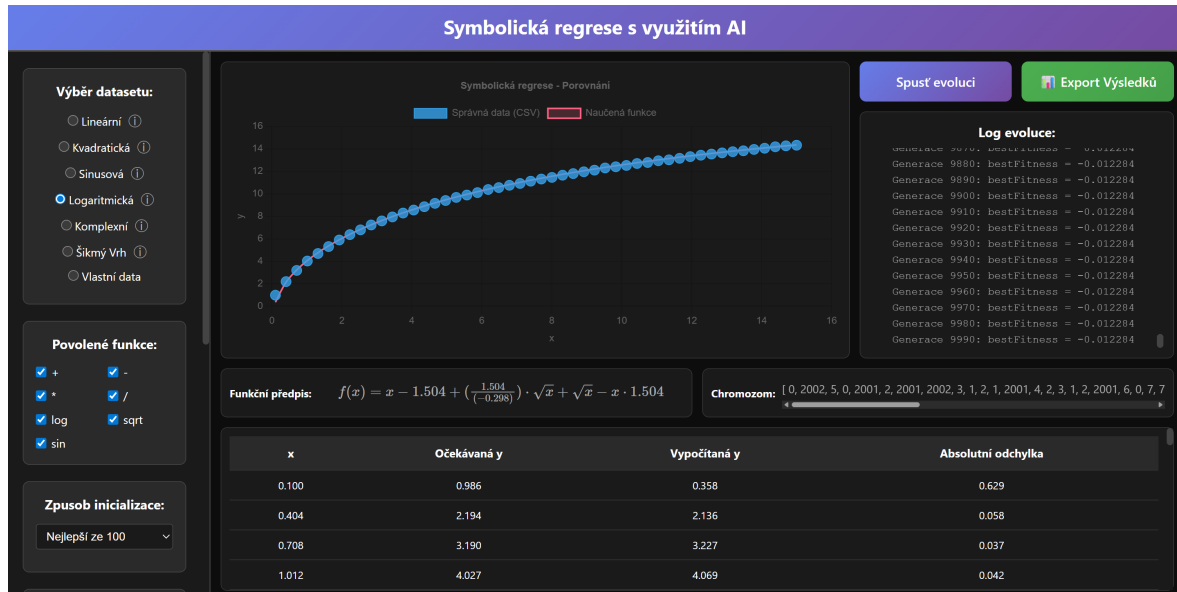
Sinusový dataset



Obrázek C.3: Rekonstrukce sinusové funkce

- **Cílový vzorec:** $y = 6 \cdot \sin(2x + 1)$
- **Nalezený vzorec:** $f(x) = \frac{\sin(\frac{x \cdot x}{0.164} + x + x)}{0.164}$
- **Po algebraické simplifikaci:** $f(x) = 6,098 \cdot \sin(2x + 1)$
- **Finální hodnota MSE:** 0,0058

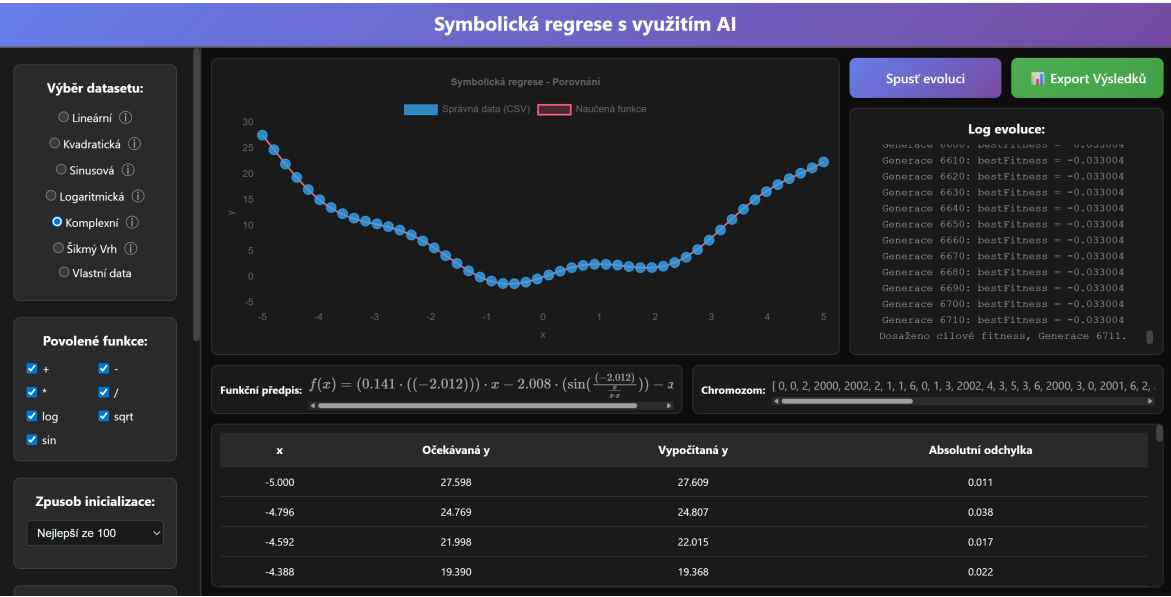
Logaritmický dataset



Obrázek C.4: Rekonstrukce logaritmické funkce.

- **Cílový vzorec:** $y = 5 \cdot \log(x + 1) + 0,5$
- **Nalezený vzorec:** $f(x) = x - 1,504 + \left(\frac{1,504}{-0,298}\right) \cdot \sqrt{x} + \sqrt{x} - x \cdot 1,504$
- **Po algebraické simplifikaci:** $f(x) = -0,504x - 4,047\sqrt{x} - 1,504$
- **Finální hodnota MSE:** 0,0123

Komplexní dataset



Obrázek C.5: Rekonstrukce komplexního datasetu

- **Cílový vzorec:** $y = x^2 + 2 \cdot \sin(2x) - 0,3x$
- **Nalezený vzorec:** $f(x) = (0,141 \cdot (-2,012)) \cdot x - 2,008 \cdot (\sin(\frac{-2,012}{x \cdot x})) - x$
- **Po algebraické simplifikaci:** $f(x) = -1,284x + 2,008 \cdot \sin(\frac{2,012}{x^2})$
- **Finální hodnota MSE:** 0,0040

Souhrnná srovnávací tabulka

V tabulce C.1 je uveden přehled rychlosti konvergence a výsledná přesnost vyjádřená pomocí střední kvadratické chyby (MSE).

Dataset	Generace pro shodu	MSE
Lineární	12	0,0026
Kvadratický	266	0,0081
Sinusový	1393	0,0058
Logaritmický	10 000	0,0123
Komplexní	6711	0,0040

Tabulka C.1: Srovnání rychlosti konvergence a přesnosti pro jednotlivé datasety při optimálním nastavení parametrů.